

Carbon Accounting in Julia

Scope 1, 2 and 3 · audit trails and security · data parsing and normalisation · calculation engines and emission-factor libraries · integration and connectivity · a neural soft sensor for the annual analyzer campaign · anomaly detection

Pluto notebook printout · 11 notebooks · generated 2026-09-21 12:47 UTC

Everything in this document is the *output of the notebooks as they were executed*: the build runs each notebook headlessly, captures its cells and renders them here.

All data is synthetic and generated deterministically by `scripts/make_data.jl`. Emission factors are published defaults — replace them with a licensed factor set before any real disclosure.

Contents

1. **00_Overview_and_Notation**
2. **01_Data_Parsing_and_Normalization**
3. **02_Integration_and_Connectivity**
4. **03_Emission_Factor_Libraries**
5. **04_Scope1_Direct_Emissions**
6. **05_Scope2_Purchased_Energy**
7. **06_Scope3_Value_Chain**
8. **07_SoftSensor_Neural_Net**
9. **08_Anomaly_Detection**
10. **09_Audit_Trail_and_Security**
11. **10_Inventory_Rollup_Uncertainty_Disclosure**

00_Overview_and_Notation

19 cells · executed in 28.8 s

```
begin
    using CarbonAccounting, CSV, DataFrames, Dates, Statistics, Printf
    using Plots
    default(fmt=:svg, legend=:topleft, size=(760, 420), dpi=120)
    datadir(parts...) = CarbonAccounting.datadir(parts...)
end
```

datadir (generic function with 1 method)

```
md"""
# Carbon accounting in Julia – platform overview

**Scope 1, 2 and 3 · audit trails and security · parsing and normalisation ·
calculation engines and emission-factor libraries · integration and connectivity ·
a neural soft sensor for the annual analyzer campaign · anomaly detection.**

The repository is a Julia package (`src/CarbonAccounting.jl`) plus eleven Pluto
notebooks that walk through it, plus a build pipeline that turns the notebooks into
the PDF you are reading.

| module | responsibility | notebook |
|---|---|---|
| `notation.jl` | symbolic legend, GWP sets,  $\Sigma_i$  | **00** |
| `ingest.jl` | parsing, normalisation, quarantine, lineage | **01** |
| `connect.jl` | connectors, incremental sync, idempotency | **02** |
| `factors.jl` | versioned emission-factor library | **03** |
| `engines.jl` | Scope 1 / 2 / 3 calculation engines | **04, 05, 06** |
| `softsensor.jl` | neural soft sensor + campaign design | **07** |
| `anomaly.jl` | EWMA, CUSUM,  $T^2$ , SPE, autoencoder | **08** |
| `security.jl` | RBAC, hash-chained audit trail, sealing | **09** |
| `uncertainty.jl`, `report.jl` | GUM + Monte Carlo, DQI, disclosure, exports | **10** |

Three design rules run through every file:

1. The code carries the equation. Every quantity keeps its mathematical symbol
( $\dot{Q}_i$ ,  $EF_i$ ,  $\hat{\sigma}$ ,  $W^{(1)}$ ,  $\hat{f}_i$ ,  $h_n$ ), so a cell can be read next to the formula
it implements.
2. A number is never reported without its qualification. Provenance (source
hash), data-quality class DQ, factor version and uncertainty travel with each row.
3. Nothing is silently repaired. Rows that cannot be interpreted are quarantined
with a reason; every state change lands in a hash-chained ledger.
"""
```

Carbon accounting in Julia — platform overview

Scope 1, 2 and 3 · audit trails and security · parsing and normalisation · calculation engines and emission-factor libraries · integration and connectivity · a neural soft sensor for the annual analyzer campaign · anomaly detection.

The repository is a Julia package (src/CarbonAccounting.jl) plus eleven Pluto notebooks that walk through it, plus a build pipeline that turns the notebooks into the PDF you are reading.

module	responsibility	notebook
notation.jl	symbolic legend, GWP sets, Σ_i	00
ingest.jl	parsing, normalisation, quarantine, lineage	01
connect.jl	connectors, incremental sync, idempotency	02
factors.jl	versioned emission-factor library	03
engines.jl	Scope 1 / 2 / 3 calculation engines	04, 05, 06
softsensor.jl	neural soft sensor + campaign design	07
anomaly.jl	EWMA, CUSUM, T^2 , SPE, autoencoder	08
security.jl	RBAC, hash-chained audit trail, sealing	09
uncertainty.jl, report.jl	GUM + Monte Carlo, DQI, disclosure, exports	10

Three design rules run through every file:

- 1. **The code carries the equation.** Every quantity keeps its mathematical symbol ($\dot{Q}_i$, EF_i , $\hat{\sigma}$, $W^{(1)}$, $\hat{f}_i$, h_n), so a cell can be read next to the formula it implements.
- 2. **A number is never reported without its qualification.** Provenance (source hash), data-quality class DQ, factor version and uncertainty travel with each row.

3. **Nothing is silently repaired.** Rows that cannot be interpreted are quarantined with a reason; every state change lands in a hash-chained ledger.

```
md"""
## 1. The notation legend is part of the code

`NOTATION` is a table of `(symbol, LaTeX, meaning, unit, area)`; `check_notation()`
asserts that every symbol in it is a legal Julia identifier, so documentation and
source cannot drift apart.
"""
```

1. The notation legend is part of the code

NOTATION is a table of (symbol, LaTeX, meaning, unit, area); check_notation() asserts that every symbol in it is a legal Julia identifier, so documentation and source cannot drift apart.

length(NOTATION), NOTATION_AREAS, check_notation()

1	90																				
2	<table><tr><td>1</td><td>"boundary"</td></tr><tr><td>2</td><td>"activity"</td></tr><tr><td>3</td><td>"factors"</td></tr><tr><td>4</td><td>"scope1"</td></tr><tr><td>5</td><td>"scope2"</td></tr><tr><td>6</td><td>"scope3"</td></tr><tr><td>7</td><td>"uncertainty"</td></tr><tr><td>8</td><td>"softsensor"</td></tr><tr><td>9</td><td>"anomaly"</td></tr><tr><td>10</td><td>"audit"</td></tr></table>	1	"boundary"	2	"activity"	3	"factors"	4	"scope1"	5	"scope2"	6	"scope3"	7	"uncertainty"	8	"softsensor"	9	"anomaly"	10	"audit"
	1	"boundary"																			
	2	"activity"																			
	3	"factors"																			
	4	"scope1"																			
	5	"scope2"																			
	6	"scope3"																			
	7	"uncertainty"																			
	8	"softsensor"																			
	9	"anomaly"																			
10	"audit"																				
3																					

Markdown.parse(notations_markdown(area="activity"))

Symbol	LaTeX	Meaning	Unit	Area
$\dot{Q}_i$	$\dot{Q}_i$	activity rate of source i: energy or mass flow per period	GJ, t, kWh	activity
$\dot{m}_i$	$\dot{m}_i$	mass flow of fuel i ($\dot{m} = \dot{n} \cdot M$: mole flow $\times$ molar mass)	t/ $\mathcal{P}$	activity
$\dot{n}_i$	$\dot{n}_i$	molar flow of species i	kmol/ $\mathcal{P}$	activity
$\dot{V}_i$	$\dot{V}_i$	gas volumetric flow, normal conditions (0 °C, 1.01325 bar)	m ³ (n)/ $\mathcal{P}$	activity
OF _i	OF _i	oxidation factor: carbon actually oxidised / carbon input (IPCC)	—	activity
η_b	η_b	boiler or combustion efficiency	—	activity
DRE	DRE	destruction & removal efficiency of a flare / abatement device	—	activity
F _i	F _i	load or utilisation factor of equipment i	—	activity
h _{op}	h _{op}	operating hours of a source inside $\mathcal{P}$	h/ $\mathcal{P}$	activity
LHV _i	LHV _i	lower heating value of fuel i	GJ/t	activity
CC _i	CC _i	carbon content of fuel i	tC/t	activity
ΔP_i	ΔP_i	leak-driving pressure differential of source i	bar	activity

Markdown.parse(notation_markdown(area="softsensor"))

Symbol	LaTeX	Meaning	Unit	Area
$\mathbf{x}$	$\vec{x}$	feature vector of continuous DCS/SCADA signals	mixed	softsensor
y	y	target variable, measured only during the analyzer campaign	mixed	softsensor
$\hat{y}$	$\hat{y}$	soft-sensor prediction, $\hat{y} = f_{\theta}(\mathbf{x})$	mixed	softsensor
θ	$\vec{\theta}$	all trainable parameters $\{W^{(l)}, b^{(l)}\}$	—	softsensor
$W^{(l)}$	$W^{(l)}$	weight matrix of layer l , $W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$	—	softsensor
$b^{(l)}$	$b^{(l)}$	bias vector of layer l	—	softsensor
$z^{(l)}$	$z^{(l)}$	pre-activation, $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$	—	softsensor
$a^{(l)}$	$a^{(l)}$	activation, $a^{(l)} = g(z^{(l)})$	—	softsensor
$\delta^{(l)}$	$\delta^{(l)}$	error signal, $\delta^{(l)} = \partial L / \partial z^{(l)}$ (back-propagation)	—	softsensor
η	η	learning rate of the optimiser	—	softsensor
β_1, β_2	β_1, β_2	Adam decay rates for the 1st / 2nd moment	—	softsensor
$\hat{m}, \hat{v}$	$\hat{m}, \hat{v}$	bias-corrected Adam moment estimates	—	softsensor
λ_{L2}	λ_{L2}	L2 weight-decay coefficient (ridge penalty $\Sigma \ W\ ^2$)	—	softsensor
L	L	loss, e.g. Gaussian negative log-likelihood	—	softsensor
$\hat{\sigma}$	$\hat{\sigma}$	estimated residual standard deviation (aleatoric noise)	mixed	softsensor
N_{camp}	N_{camp}	campaign samples, 7 d · 24 h = 168 per source	—	softsensor
D^2_M	D^2_M	Mahalanobis distance of a sample from the campaign cloud	—	softsensor
ε	ε	numerical floor (division by zero / log 0 protection)	—	softsensor

```
md"""
## 2. Units are types, so `E = Q · EF` cannot be wrong by accident

A factor is converted into the unit the activity is measured in — and refused when
the dimension does not match:

| activity | factor | result |
|---|---|---|
| `1250 MWh` | `0.42 kgCO2e/kWh` | 525 tCO2e |
| `1250 MWh` | `0.42 kgCO2e/GJ` | 1.89 tCO2e — MWh → GJ, faithfully converted |
| `1250 MWh` | `0.42 kgCO2e/t` | DimensionMismatch — energy vs mass |

Canonical units: mass `t`, CO2e `tCO2e`, energy `GJ`, volume `m³` (normal
conditions), distance `km`, transport work `t·km`, money `USD2024`, temperature `K`.
"""
```

2. Units are types, so E = Q · EF cannot be wrong by accident

A factor is converted into the unit the activity is measured in — and refused when the *dimension* does not match:

activity	factor	result
1250 MWh	0.42 kgCO ₂ e/kWh	525 tCO ₂ e
1250 MWh	0.42 kgCO ₂ e/GJ	1.89 tCO ₂ e — MWh → GJ, faithfully converted
1250 MWh	0.42 kgCO ₂ e/t	DimensionMismatch — energy vs mass

Canonical units: mass t, CO₂e tCO₂e, energy GJ, volume m³ (normal conditions), distance km, transport work t·km, money USD2024, temperature K.

```
begin
    inputs = ["1,250 MWh", "1,250 kWh", "3,412 Btu", "1 toe", "2,204.6 lb", "1,000 scf"]
    q = parse_quantity.(inputs)
    DataFrame(as_written=inputs, value_canonical=val_canon.(q),
              canonical_unit=[CANONICAL[dimension(x)] for x in q])
end
```

1	"1,250 MWh"	4500.0	"GJ"
2	"1,250 kWh"	4.5	"GJ"
3	"3,412 Btu"	0.00359985	"GJ"
4	"1 toe"	41.868	"GJ"
5	"2,204.6 lb"	0.99999	"t"
6	"1,000 scf"	28.3168	"m³"

```
begin
    Q̇ᵢ = q"1250 MWh" # the activity as the utility reports it
    EFᵢ = q"0.42 kgCO₂e/kWh" # the factor as the library holds it
    (Eᵢ = emission(Q̇ᵢ, EFᵢ),
     same_energy_in_GJ = emission(q"1250 MWh", q"0.42 kgCO₂e/GJ"),
     refused = try
         emission(q"1250 MWh", q"0.42 kgCO₂e/t") # energy × mass factor: refused
     catch e
         sprint(showerror, e)
     end)
end
```

Eᵢ	525.0 tCO₂e
same_energy_in_GJ	1.89 tCO₂e
refused	"DimensionMismatch: activity «MWh» (energy) does not match factor denominator t (mass)"

```
md"""
## 3. The GWP conversion

`E_CO₂e = E_CO₂ + GWP_CH₄ · E_CH₄ + GWP_N₂O · E_N₂O`. The set used is part of the
disclosure: AR6 separates fossil from biogenic methane (29.8 vs 27.0 over
100 years), so a biogenic slip valued with the fossil factor is 10 % too high.
"""
```

3. The GWP conversion

$E_{CO_2e} = E_{CO_2} + GWP_{CH_4} \cdot E_{CH_4} + GWP_{N_2O} \cdot E_{N_2O}$. The set used is part of the disclosure: AR6 separates **fossil** from **biogenic** methane (29.8 vs 27.0 over 100 years), so a biogenic slip valued with the fossil factor is 10 % too high.

```
begin
    t = 1.0 # one tonne of each gas
    DataFrame(gas=["CH₄ fossil", "CH₄ biogenic", "N₂O", "CO₂"],
              AR4=[co2e(; CH₄=t, gwp=GWP_AR4), co2e(; CH₄=t, gwp=GWP_AR4, fossil=false),
                  co2e(; N₂O=t, gwp=GWP_AR4), t],
              AR5=[co2e(; CH₄=t, gwp=GWP_AR5), co2e(; CH₄=t, gwp=GWP_AR5, fossil=false),
                  co2e(; N₂O=t, gwp=GWP_AR5), t],
              AR6=[co2e(; CH₄=t), co2e(; CH₄=t, fossil=false), co2e(; N₂O=t), t])
end
```

1	"CH₄ fossil"	25.0	30.0	29.8
2	"CH₄ biogenic"	25.0	28.0	27.0
3	"N₂O"	298.0	265.0	273.0
4	"CO₂"	1.0	1.0	1.0

```
md"""
## 4. The platform end to end

The remaining cells run the whole chain on the repository's synthetic portfolio –
six sites in five countries, one reporting year – in a few seconds, so that the
following notebooks can go deep on one link at a time:

    raw exports → ingest → normalise → factors → engines → uncertainty → disclosure
                  \ soft sensor \ anomaly detection \ audit trail ↗
"""
```

4. The platform end to end

The remaining cells run the whole chain on the repository's synthetic portfolio — six sites in five countries, one reporting year — in a few seconds, so that the following notebooks can go deep on one link at a time:

```
raw exports → ingest → normalise → factors → engines → uncertainty → disclosure
              \ soft sensor \ anomaly detection \ audit trail ↗
```

```
begin
    paths = [datadir("raw", f) for f in
              ["activity_erp_2024.csv", "electricity_utility_2024.csv", "procurement_2024.csv",
               "logistics_travel_2024.csv", "waste_2024.csv"]]
    records = ingest_files(paths; entity_default="AcmeIndustrial_SA").records
    inventory = compute_inventory(records; consolidation=:operational, date=Date(2024, 12, 31))
    inventory
end
```

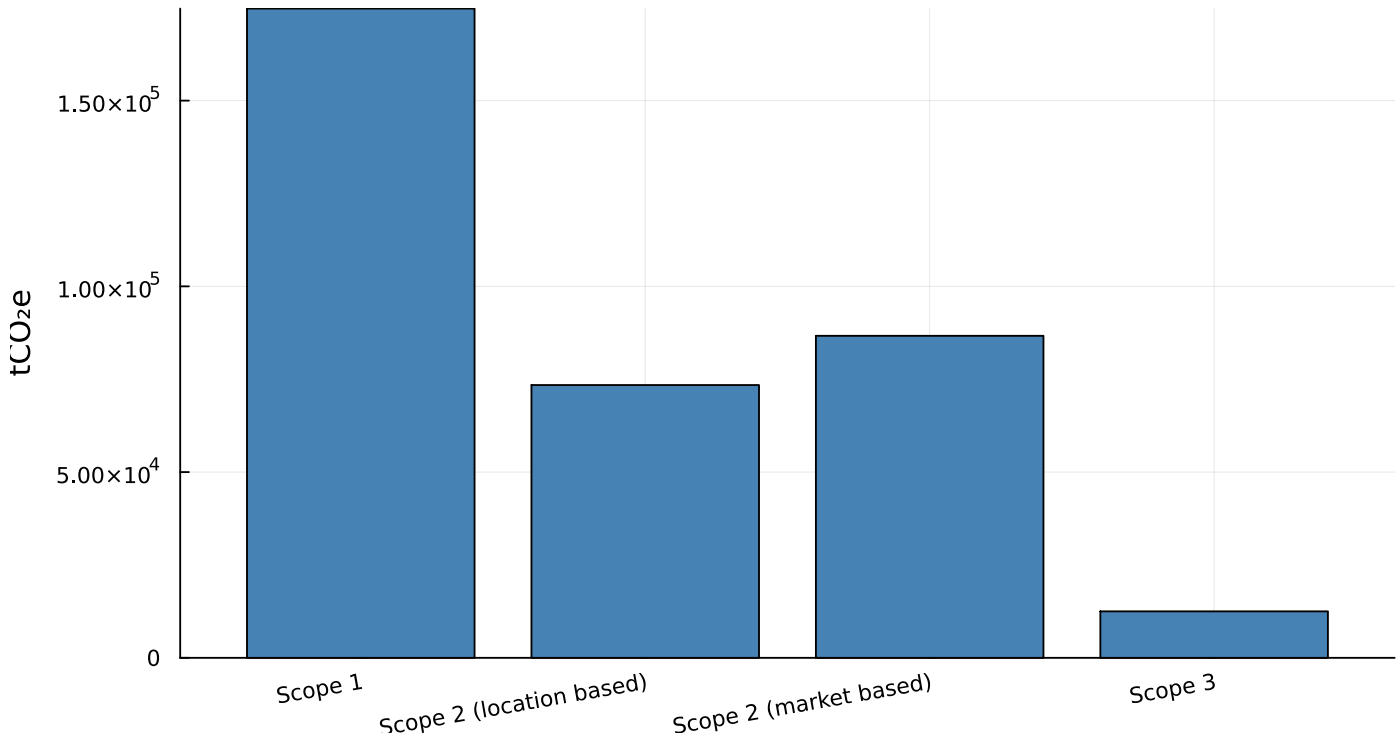
```
InventoryResult [operational]
E_Scope1      = 174781.393 tCO₂e
E_Scope2 loc  =  73433.265 tCO₂e
E_Scope2 mkt  =  86683.228 tCO₂e
E_Scope3      =  12509.686 tCO₂e
E_total       = 260724.344 tCO₂e (445 rows)
```

```
contribution_table(inventory)
```

1	"Scope 1"	1.74781e5	0.670369
2	"Scope 2 (location based)"	73433.3	0.281651
3	"Scope 2 (market based)"	86683.2	0.332471
4	"Scope 3"	12509.7	0.0479805

```
begin
    contrib = contribution_table(inventory)
    plt = bar(contrib.component, contrib.E,
              title="Greenhouse-gas inventory 2024 (operational control)",
              ylabel="tCO2e", xrotation=10, legend=false,
              color=[:steelblue :seagreen :darkseagreen :darkorange])
    save_figure(plt, "00_contribution"; saver=savefig)
    plt
end
```

Greenhouse-gas inventory 2024 (operational control)



```
md"""
## 5. How to read the rest

* every notebook activates the same environment and reads the same files, so each can
  be run on its own;
* cells keep the symbols of the equations: where a cell reads `E_Scope1 = Σi(...)`, the
  expression between `Σi` and the closing parenthesis is the summand of the published
  formula;
* the last cell of each notebook states what was verified, not merely what was
  computed – the same sentence appears in the PDF printout.

The PDF in `docs/` is produced by `julia --project=. scripts/build_pdf.jl`, which runs
these notebooks headlessly, renders the cell outputs to HTML and prints that to PDF
with headless Chrome.
"""
```

5. How to read the rest

- every notebook activates the same environment and reads the same files, so each can be run on its own;
- cells keep the symbols of the equations: where a cell reads $E_{\text{Scope1}} = \Sigma_i(\dots)$, the expression between Σ_i and the closing parenthesis is the summand of the published formula;
- the last cell of each notebook states what was **verified**, not merely what was computed — the same sentence appears in the PDF printout.

The PDF in docs/ is produced by `julia --project=. scripts/build_pdf.jl`, which runs these notebooks headlessly, renders the cell outputs to HTML and prints that to PDF with headless Chrome.

```
md"""
---
*Notebook 00 of the Carbon Accounting platform · full documentation in `README.md`.*
"""
```

Notebook 00 of the Carbon Accounting platform · full documentation in README.md.

```
begin
    # last cell: state what was *verified*, not merely what was computed
    println("notation symbols checked : ", length(NOTATION), " (invalid: ", check_notation(), ")")
    println("activity records          : ", length(records))
    println("inventory total           : ", round(inventory.E_total; digits=1), " tCO2e over ",
            length(inventory.rows), " calculated rows")
    println("Notebook 00 – the legend and the source code are consistent, and the platform runs end to
end.")
end
```

01_Data_Parsing_and_Normalization

20 cells · executed in 19.04 s

```
begin
    using CarbonAccounting, CSV, DataFrames, Dates, Statistics, Printf
    using Plots
    default(fmt=:svg, legend=:topleft, size=(760, 420), dpi=120)
    datadir(parts...) = CarbonAccounting.datadir(parts...)
end
```

datadir (generic function with 1 method)

```
md"""
# 01 · Data parsing and normalisation

Raw activity data arrives from meters, ERP extracts, fuel invoices and utility
portals: mixed delimiters, units glued to the value ("1,234.5 MWh"), decimal
commas, several date conventions, multi-language headers, duplicated rows.

`src/ingest.jl` turns that into typed `ActivityRecord`s carrying a canonical
unit per dimension (`t`, `GJ`, `m³`, `kWh`, `USD2024`, `t·km`), an explicit period
 $\mathcal{P} = [\tau_0, \tau_1]$ , a data-quality class  $DQ \in \{1,2,3,4\}$  and full lineage — and it
quarantines every row it cannot interpret, with the reason, instead of silently
dropping or coercing it.
"""
```

01 · Data parsing and normalisation

Raw activity data arrives from meters, ERP extracts, fuel invoices and utility portals: mixed delimiters, units glued to the value ("1,234.5 MWh"), decimal commas, several date conventions, multi-language headers, duplicated rows.

`src/ingest.jl` turns that into typed `ActivityRecords` carrying a **canonical unit** per dimension (`t`, `GJ`, `m³`, `kWh`, `USD2024`, `t·km`), an explicit period $\mathcal{P} = [\tau_0, \tau_1]$, a **data-quality class** $DQ \in \{1,2,3,4\}$ and full **lineage** — and it *quarantines* every row it cannot interpret, with the reason, instead of silently dropping or coercing it.

```
md"""### The raw export, exactly as it arrived — semicolons, decimal commas, German headers"
```

The raw export, exactly as it arrived — semicolons, decimal commas, German headers

```
begin
    f = datadir("raw", "activity_erp_2024.csv")
    println.(first(eachline(f), 8))
    f
end
```

"C:\\Users\\inter\\ca_acct_repo\\data\\raw\\activity_erp_2024.csv"

```
md"""
### Scalar parsers

`parse_number` accepts `1.234,5` as well as `1,234.5`; `parse_period` resolves
`2024-Q1`, `2024-03`, `2024` and `01.03.2024-31.03.2024` into  $\mathcal{P} = [\tau_0, \tau_1]$ ; the
`dayfirst` flag resolves `01/02/2024` explicitly instead of guessing.
"""
```

Scalar parsers

`parse_number` accepts `1.234,5` as well as `1,234.5`; `parse_period` resolves `2024-Q1`, `2024-03`, `2024` and `01.03.2024-31.03.2024` into $\mathcal{P} = [\tau_0, \tau_1]$; the `dayfirst` flag resolves `01/02/2024` explicitly instead of guessing.

```
begin
    nums = ["1.234,5", "1,234.5", "(1 250,75)", "12,5", "3.141,59"]
    dates = ["2024-03-31", "31/03/2024", "31.03.2024", "2024-03"]
    (numeric=[parse_number(s) for s in nums],
     dates=[parse_date(s) for s in dates],
     periods=[string(parse_period(s)) for s in
               ["2024-Q1", "2024-03", "2024", "01.03.2024-31.03.2024"]])
end
```

numeric	1	1234.5
	2	1234.5
	3	-1250.75
	4	12.5
	5	3141.59
dates	1	2024-03-31
	2	2024-03-31
	3	2024-03-31
	4	2024-03-01
periods	1	"(Dates.Date(\"2024-01-01\"), Dates.Date(\"2024-03-31\"))"
	2	"(Dates.Date(\"2024-03-01\"), Dates.Date(\"2024-03-31\"))"
	3	"(Dates.Date(\"2024-01-01\"), Dates.Date(\"2024-12-31\"))"
	4	"(Dates.Date(\"2024-03-01\"), Dates.Date(\"2024-03-31\"))"

```
md"""
```

```
### Ingest the five exports
```

Every file is hashed (SHA-256) and — because a ledger and an actor are passed — the ingestion itself is written to the audit trail, so the *provenance* of every figure is provable rather than asserted.

```
"""
```

Ingest the five exports

Every file is hashed (SHA-256) and — because a ledger and an actor are passed — the ingestion itself is written to the audit trail, so the *provenance* of every figure is provable rather than asserted.

```
begin
    ledger = AuditLedger(hmac_key=Vector{UInt8}(codeunits("notebook-01")))
    analyst = Actor("u001", "A. Analyst", :analyst)
    files = ["activity_erp_2024.csv", "electricity_utility_2024.csv", "procurement_2024.csv",
            "logistics_travel_2024.csv", "waste_2024.csv"]
    report = ingest_files([datadir("raw", f) for f in files];
                          entity_default="AcmeIndustrial_SA", ledger=ledger, actor=analyst,
                          context="notebook 01 - ingest run")

    report
end
```

```
IngestReport: 373 records, 2 quarantined, 5 file(s)
activity_erp_2024.csv sha256=153a1d4a953e... rows=177
electricity_utility_2024.csv sha256=5493b5ea9f79... rows=72
procurement_2024.csv sha256=827f861091b4... rows=42
logistics_travel_2024.csv sha256=a10fc2eb64d6... rows=66
waste_2024.csv sha256=675305a28ae5... rows=18
```

```
md"""
```

```
### What was quarantined, and why
```

```
The quarantine is the audit-friendly alternative to silent repair: the rows of the
demo that cannot be interpreted appear here with their reason.
"""
```

What was quarantined, and why

The quarantine is the audit-friendly alternative to silent repair: the rows of the demo that cannot be interpreted appear here with their reason.

```
begin
    q = quarantine_dataframe(report.quarantined)
    isempty(q) ? q : combine(groupby(q, :reason), nrow => :rows)
end
```

1	"unknown source type «Wundermittel» (no matching SourceRule)"	1
2	"unknown unit «Epstein-Barrel» — register it in _register_units! (src/units.jl)"	1

```
begin
    records = report.records
    df = records_dataframe(records)
    select(df, :rec_id, :site, :source_type, :item, :τ, :value, :unit, :Q_canon,
           :unit_canon, :DQ, :source, :source_hash)
end
```

1	"activity_erp_2024.csv:2"	"SAO-01"	"natural_gas"	"Erdgas Kessel 1"	2024-01-01	30184.0	"GJ"	30184.0	"GJ"	2	"activity_erp_2024.csv"	"153a1d4a953e"
2	"activity_erp_2024.csv:3"	"SAO-01"	"diesel_fleet"	"Diesel Gerador"	2024-01-01	5595.0	"L"	5.595	"m³"	2	"activity_erp_2024.csv"	"153a1d4a953e"
3	"activity_erp_2024.csv:4"	"SAO-01"	"natural_gas"	"Erdgas Kessel 1"	2024-02-01	27259.0	"GJ"	27259.0	"GJ"	2	"activity_erp_2024.csv"	"153a1d4a953e"
4	"activity_erp_2024.csv:5"	"SAO-01"	"diesel_fleet"	"Diesel Gerador"	2024-02-01	6416.0	"L"	6.416	"m³"	2	"activity_erp_2024.csv"	"153a1d4a953e"
5	"activity_erp_2024.csv:6"	"SAO-01"	"natural_gas"	"Erdgas Kessel 1"	2024-03-01	27010.0	"GJ"	27010.0	"GJ"	2	"activity_erp_2024.csv"	"153a1d4a953e"
6	"activity_erp_2024.csv:7"	"SAO-01"	"diesel_fleet"	"Diesel Gerador"	2024-03-01	5481.0	"L"	5.481	"m³"	2	"activity_erp_2024.csv"	"153a1d4a953e"
7	"activity_erp_2024.csv:8"	"SAO-01"	"natural_gas"	"Erdgas Kessel 1"	2024-04-01	22205.0	"GJ"	22205.0	"GJ"	2	"activity_erp_2024.csv"	"153a1d4a953e"
8	"activity_erp_2024.csv:9"	"SAO-01"	"diesel_fleet"	"Diesel Gerador"	2024-04-01	5874.0	"L"	5.874	"m³"	2	"activity_erp_2024.csv"	"153a1d4a953e"
9	"activity_erp_2024.csv:10"	"SAO-01"	"natural_gas"	"Erdgas Kessel 1"	2024-05-01	18750.0	"GJ"	18750.0	"GJ"	2	"activity_erp_2024.csv"	"153a1d4a953e"
10	"activity_erp_2024.csv:11"	"SAO-01"	"diesel_fleet"	"Diesel Gerador"	2024-05-01	5944.0	"L"	5.944	"m³"	2	"activity_erp_2024.csv"	"153a1d4a953e"
more												
373	"waste_2024.csv:19"	"SHA-03"	"waste"	"incineration"	2024-01-01	268.0	"t"	268.0	"t"	3	"waste_2024.csv"	"675305a28ae5"

```
md"""
### Normalisation is a dimensional step, not a cosmetic one

`1.000 L` of diesel, `4,200 kg` of the same fuel and `57 GJ` of heat all land in the
canonical unit of their dimension. The engines downstream only ever see canonical
units, which is why a unit mix-up cannot survive into the total.
"""
```

Normalisation is a *dimensional* step, not a cosmetic one

1.000 L of diesel, 4,200 kg of the same fuel and 57 GJ of heat all land in the canonical unit of their dimension. The engines downstream only ever see canonical units, which is why a unit mix-up cannot survive into the total.

```
combine(groupby(records_dataframe(records), [:scope, :unit, :unit_canon]),
         nrow => :records, :Q_canon => sum => :activity)
```

1	"scope1"	"GJ"	"GJ"	73	1.55818e6
2	"scope1"	"L"	"m ³ "	72	368.14
3	"scope1"	"t"	"t"	12	326208.0
4	"scope1"	"kg"	"t"	6	0.868
5	"scope1"	"m ³ (n)"	"m ³ "	6	1.73591e7
6	"scope1"	"component"	"unit"	6	13382.0
7	"scope2"	"MWh"	"GJ"	39	331272.0
8	"scope2"	"kWh"	"GJ"	33	2.69459e5
9	"scope3"	"BRL"	"USD2024"	15	809969.0
10	"scope3"	"EUR"	"USD2024"	19	6.09814e6
11	"scope3"	"CNY"	"USD2024"	5	2.56258e5
12	"scope3"	"ZAR"	"USD2024"	3	82537.9
13	"scope3"	"t-km"	"t-km"	30	6.20511e7
14	"scope3"	"p-km"	"t-km"	36	3.71639e7
15	"scope3"	"t"	"t"	18	7332.0

```
md"""
### Reporting controls before any calculation

`validate_records` applies the checks an auditor asks for first: non-positive
activity, missing entity, periods outside the reporting year, proxy-quality rows,
implausible magnitudes. `:error` rows must be corrected; `:info` rows are those to
refine with supplier data.
"""
```

Reporting controls before any calculation

`validate_records` applies the checks an auditor asks for first: non-positive activity, missing entity, periods outside the reporting year, proxy-quality rows, implausible magnitudes. `:error` rows must be corrected; `:info` rows are those to refine with supplier data.

```

begin
    issues = validate_records(records; year=2024)
    (by_severity=combine(groupby(DataFrame(issues), :severity), nrow => :n),
     examples=[(i.rec_id, i.severity, i.message) for i in first(issues, 4)])
end

```

by_severity	1	:info	66
	2	:error	6
examples	1	1	"activity_erp_2024.csv:30"
		2	:info
		3	"proxy/spend-based estimate (DQ 4) — refine with supplier data"
	2	1	"activity_erp_2024.csv:59"
		2	:info
		3	"proxy/spend-based estimate (DQ 4) — refine with supplier data"
	3	1	"activity_erp_2024.csv:88"
		2	:info
		3	"proxy/spend-based estimate (DQ 4) — refine with supplier data"
	4	1	"activity_erp_2024.csv:117"
		2	:info
		3	"proxy/spend-based estimate (DQ 4) — refine with supplier data"

```

md"""
### Data quality weighted by emissions, and the lineage of every file

DQ 1 is supplier- or meter-specific, DQ 4 a proxy or spend-based estimate. The
weighted table shows where the inventory's soft spots are, ranked by the tonnes they
carry - and `report.files` holds the SHA-256 of every file ingested.
"""

```

Data quality weighted by emissions, and the lineage of every file

DQ 1 is supplier- or meter-specific, DQ 4 a proxy or spend-based estimate. The weighted table shows where the inventory's soft spots are, ranked by the tonnes they carry — and `report.files` holds the SHA-256 of every file ingested.

```

begin
    inventory = compute_inventory(records; consolidation=:operational, date=Date(2024, 12, 31))
    dq1_table(inventory.rows)
end

```

1	"electricity"	1.60116e5	0.460889	2.0	0.11709	"B — good, average data with documented source"
2	"natural_gas"	97225.2	0.279859	2.0	0.708378	"B — good, average data with documented source"
3	"flaring"	38701.1	0.1114	2.0	0.253574	"B — good, average data with documented source"
4	"process_nitric_acid"	34892.4	0.100437	2.0	0.402244	"B — good, average data with documented source"
5	"purchased_goods_spend"	4162.14	0.0119806	4.0	0.474342	"D — low, screening estimate to be refined"
6	"employee_commuting"	4089.19	0.0117706	3.0	0.23065	"C — fair, industry average or proxy"
7	"refrigerant"	1812.38	0.00521688	2.0	0.108628	"B — good, average data with documented source"
8	"waste"	1724.34	0.00496344	3.0	0.359212	"C — fair, industry average or proxy"
9	"upstream_transport"	1349.18	0.00388358	3.0	0.206128	"C — fair, industry average or proxy"
10	"diesel_fleet"	993.904	0.00286092	2.0	0.0519615	"B — good, average data with documented source"
11	"process_cement"	956.923	0.00275447	2.0	0.0583095	"B — good, average data with documented source"
12	"downstream_transport"	602.539	0.00173439	3.0	0.203151	"C — fair, industry average or proxy"
13	"business_travel_air"	327.079	0.000941484	3.0	0.211896	"C — fair, industry average or proxy"
14	"business_travel_land"	255.221	0.000734644	3.0	0.165529	"C — fair, industry average or proxy"
15	"ldar"	199.392	0.000573942	4.0	0.636396	"D — low, screening estimate to be refined"

```
DataFrame(file=basename.([string(f["path"]) for f in report.files]),
          rows=[f["rows"] for f in report.files],
          delimiter=[f["delim"] for f in report.files],
          sha256=[first(string(f["sha256"]), 16) for f in report.files])
```

1	"activity_erp_2024.csv"	177	","	"153a1d4a953e9b24"
2	"electricity_utility_2024.csv"	72	","	"5493b5ea9f79c11e"
3	"procurement_2024.csv"	42	","	"827f861091b4e9e7"
4	"logistics_travel_2024.csv"	66	","	"a10fc2eb64d6d6e7"
5	"waste_2024.csv"	18	","	"675305a28ae5a95f"

```
begin
    println("files ingested      : ", length(report.files))
    println("records accepted   : ", length(records))
    println("rows quarantined      : ", length(report.quarantined), " (reasons above)")
    println("validation findings : ", length(issues))
    println("audit trail          : ", chain_statement(ledger))
    println("Notebook 01 – every row is typed, dimension-checked, quality-tagged and traceable.")
end
```

02_Integration_and_Connectivity

14 cells · executed in 16.89 s

```
begin
    using CarbonAccounting, CSV, DataFrames, Dates, Statistics, Printf
    datadir(parts...) = CarbonAccounting.datadir(parts...)
end
```

datadir (generic function with 1 method)

```
md"""
# 02 · Integration and connectivity

Emissions data does not appear in a spreadsheet by magic: it is pulled from
meters, historians, ERPs, utility portals and supplier APIs. `src/connect.jl`
defines a small connector contract with the properties an audited inventory needs:

| property | mechanism |
|---|---|
| incremental sync | only rows newer than the stored high-water mark τ return (`since=...`) |
| idempotency | every row carries a deterministic key — a replayed page cannot double-count |
| resilience | transient failures retried with exponential back-off and jitter |
| provenance | the connector returns a `source_hash` and the next watermark |

Three connectors are implemented: a folder of exports, a REST/JSON API, and
in-memory rows. A mock ERP serves the demo API in-process, so this notebook runs
without a network.
"""
```

02 · Integration and connectivity

Emissions data does not appear in a spreadsheet by magic: it is pulled from meters, historians, ERPs, utility portals and supplier APIs. `src/connect.jl` defines a small connector contract with the properties an audited inventory needs:

property	mechanism
incremental sync	only rows newer than the stored high-water mark τ return (<code>since=...</code>)
idempotency	every row carries a deterministic key — a replayed page cannot double-count
resilience	transient failures retried with exponential back-off and jitter
provenance	the connector returns a <code>source_hash</code> and the next watermark

Three connectors are implemented: a **folder** of exports, a **REST/JSON API**, and in-memory rows. A mock ERP serves the demo API in-process, so this notebook runs without a network.

```
md"""### A folder connector — the 'our utility sends a CSV every month' interface"
```

A folder connector — the 'our utility sends a CSV every month' interface

```
begin
    ledger = AuditLedger(hmac_key=Vector{UInt8}(codeunits("notebook-02")))
    service = Actor("svc-etl", "ETL Service", :service)
    folder = CSVFolderConnector(datadir("raw"), r"electricity.*\.csv$"i)
    pull(folder; ledger=ledger, actor=service, context="nightly folder sync")
end
```

PullResult(72 rows, 1 page(s), 1 attempt(s), watermark=2026-09-21T11:06:26.346, sha256=b3cf18d7457f...)

```
md"""
### A REST connector against an API that misbehaves

`MockERP` serves paginated JSON with `since` filtering and injects two `503`s, so the
retry logic is demonstrated rather than asserted. The pull reports how many HTTP
attempts it needed – evidence for the audit trail.
"""
```

A REST connector against an API that misbehaves

MockERP serves paginated JSON with since filtering and injects two 503s, so the retry logic is demonstrated rather than asserted. The pull reports how many HTTP attempts it needed — evidence for the audit trail.

```
begin
    function erp_rows()
        rows = Dict{String,String}[]
        for r in CSV.File(datadir("raw", "activity_erp_2024.csv"); delim=';',
                               types=String, missingstring=String[])
            d = Dict{String,String}(string(k) => string(v) for (k, v) in pairs(r))
            d["timestamp"] = "2024-03-01T00:00:00"
            push!(rows, d)
        end
        rows
    end
    erp = MockERP(erp_rows(); port=8877, page_size=25, transients=2)
    start!(erp)
    erp
end
```

```
MockERP(port=8877, rows=177, 1 hit(s), running)
```

```
begin
    conn = RESTConnector(erp_url(erp); page_size=25, token="demo-token")
    state = SyncState()
    records, quarantined, pulled = sync!(conn, state; ledger=ledger, actor=service,
                                         context="first incremental sync")
    (pull=pulled, accepted=length(records), quarantined=length(quarantined),
     watermark=state.watermark)
end
```

pull	PullResult(177 rows, 8 page(s), 8 attempt(s), watermark=1970-01-01T00:00:00, sha256=ef8212fa3cc7...)
accepted	175
quarantined	2
watermark	1970-01-01T00:00:00

```
md"""
### Idempotency: run the same sync again

The second run returns zero new records: every row's idempotency key is already
in the sync state. This is what protects the inventory from a replayed page after a
timeout, the most common way a connector silently doubles emissions.
"""
```

Idempotency: run the same sync again

The second run returns **zero new records**: every row's idempotency key is already in the sync state. This is what protects the inventory from a replayed page after a timeout, the most common way a connector silently doubles emissions.

```
begin
    again, _, pulled2 = sync!(conn, state; ledger=ledger, actor=service,
                               context="replay after timeout")
    (replayed_rows_accepted=length(again), attempts=pulled2.attempts,
     seen_keys=length(state.seen_keys))
end
```

replayed_rows_accepted	0
attempts	8
seen_keys	177

```
begin
    # the idempotency key is a hash of the row content: stable across runs, machines and order
    k1 = idempotency_key("rest:127.0.0.1", Dict("Kategorie" => "Erdgas", "Verbrauch" => "18000"))
    k2 = idempotency_key("rest:127.0.0.1", Dict("Verbrauch" => "18000", "Kategorie" => "Erdgas"))
    (description="key is content-addressed, so field order does not matter",
     key=first(k1, 24), equal=(k1 == k2))
end
```

description	"key is content-addressed, so field order does not matter"
key	"f27d7a2085b24302f35e0de5"
equal	true

md"### The synchronisation history – one row per pull, with attempts and payload hash"

The synchronisation history — one row per pull, with attempts and payload hash

```
sync_history(state)
```

1	2026-09-21T12:40:48.501	"rest:127.0.0.1:8877"	175	8	"ef8212fa3cc7a892"
2	2026-09-21T12:40:48.848	"rest:127.0.0.1:8877"	0	8	"ef8212fa3cc7a892"

```
begin
    stop!(erp)
    println("mock API hits           : ", erp.hits)
    println("records accepted        : ", length(records))
    println("watermark                   : ", state.watermark)
    println("audit trail                    : ", chain_statement(ledger))
    println("Notebook 02 – ingestion is incremental, replayed safely and logged with its payload hash.")
end
```

03_Emission_Factor_Libraries

16 cells · executed in 16.82 s

```
begin
    using CarbonAccounting, CSV, DataFrames, Dates, Statistics, Printf
    datadir(parts...) = CarbonAccounting.datadir(parts...)
    figdir(parts...) = CarbonAccounting.figdir(parts...)
end
```

figdir (generic function with 1 method)

```
md"""
# 03 · Emission-factor libraries

A factor is not a number, it is a claim: it holds for a given activity, region,
gas and time window, somebody published it, and it carries an uncertainty. Every
record is therefore stored as an immutable `EFRecord` with

    EFi = (value, unit, gas, region, GWP set, validity window, DQ,  $\sigma/\mu$ , source, version)

and resolved for an activity by (region → GLOBAL, newest version, item match).
Changing a factor creates a new version; it never overwrites history.
"""
```

03 · Emission-factor libraries

A factor is not a number, it is a **claim**: it holds for a given activity, region, gas and time window, somebody published it, and it carries an uncertainty. Every record is therefore stored as an immutable EFRecord with

EF_i = (value, unit, gas, region, GWP set, validity window, DQ, σ/μ , source, version)

and resolved for an activity by (region → GLOBAL, newest version, item match). Changing a factor creates a **new version**; it never overwrites history.

```
begin
    lib = default_library()
    (count=length(lib), name=lib.name, version=lib.version,
     categories=length(unique(f.category for f in lib.records)),
     sources=length(unique(f.source for f in lib.records)))
end
```

count	216
name	"CarbonAccounting built-in"
version	"2026.1"
categories	39
sources	60

```
begin
    df = factors_dataframe(lib)
    first(select(df, :category, :item, :gas, :value, :unit, :region, :version, :DQ, :uncertainty), 8)
end
```

1	"natural_gas"	"natural_gas"	"CO2"	56.1	"kgCO ₂ /GJ"	"GLOBAL"	"2026.1"	2	0.03
2	"natural_gas"	"natural_gas"	"CH4"	0.001	"kgCH ₄ /GJ"	"GLOBAL"	"2026.1"	2	0.5
3	"natural_gas"	"natural_gas"	"N2O"	0.0001	"kgN ₂ O/GJ"	"GLOBAL"	"2026.1"	2	0.5
4	"fuel_oil"	"fuel_oil"	"CO2"	77.4	"kgCO ₂ /GJ"	"GLOBAL"	"2026.1"	2	0.03
5	"fuel_oil"	"fuel_oil"	"CH4"	0.003	"kgCH ₄ /GJ"	"GLOBAL"	"2026.1"	2	0.5
6	"fuel_oil"	"fuel_oil"	"N2O"	0.0006	"kgN ₂ O/GJ"	"GLOBAL"	"2026.1"	2	0.5
7	"diesel_stationary"	"diesel_stationary"	"CO2"	74.1	"kgCO ₂ /GJ"	"GLOBAL"	"2026.1"	2	0.02
8	"diesel_stationary"	"diesel_stationary"	"CH4"	0.003	"kgCH ₄ /GJ"	"GLOBAL"	"2026.1"	2	0.5

```
md"""
### Precedence: the same factor id, several claims

`lookup_factor` returns the candidates in precedence order – the requested region
first, then `GLOBAL`, then any other region, newest version inside each group. The
order is explicit, so a report can state which factor set it used.
"""
```

Precedence: the same factor id, several claims

`lookup_factor` returns the candidates in precedence order — the requested region first, then GLOBAL, then any other region, newest version inside each group. The order is explicit, so a report can state *which* factor set it used.

```

begin
# a factor that exists in several regions and versions
extras = [EFRecord("grid_DEU_2025.1", "Utility disclosure 2025", "2025.1", "electricity",
    "grid_average", "CO2e", 0.352, "kgCO2e/kWh", "DEU", "AR5",
    Date(2025, 1, 1), Date(2026, 12, 31), 2, 0.04, "utility residual disclosure", ""),
    EFRecord("grid_BRA_2025.1", "Utility disclosure 2025", "2025.1", "electricity",
    "grid_average", "CO2e", 0.098, "kgCO2e/kWh", "BRA", "AR5",
    Date(2025, 1, 1), Date(2026, 12, 31), 2, 0.06, "utility residual disclosure", "")]
lib2 = FactorLibrary(lib.name, "2026.2", vcat(lib.records, extras))
(rows=[(f.region, f.version, f.value, f.source) for f in
    lookup_factor(lib2, "electricity", "grid_average"; region="DEU", date=Date(2025, 6, 1))],
    note="DEU 2025.1 wins; the 2026.1 GLOBAL default follows")
end

```

rows	1	1	"DEU"
		2	"2025.1"
		3	0.352
		4	"utility residual disclosure"
	2	1	"DEU"
		2	"2026.1"
		3	0.38
		4	"IEA/Ember 2023 grid intensity"
	3	1	"GLOBAL"
		2	"2026.1"
		3	0.436
		4	"IEA/Ember 2023 grid intensity"
	4	1	"BRA"
		2	"2025.1"
		3	0.098
		4	"utility residual disclosure"
	5	1	"BRA"
		2	"2026.1"
		3	0.087
		4	"IEA/Ember 2023 grid intensity"
	6	1	"ZAF"
		2	"2026.1"
		3	0.95
		4	"IEA/Ember 2023 grid intensity"
	7	1	"USA"
		2	"2026.1"
		3	0.369
		4	"IEA/Ember 2023 grid intensity"
	8	1	"CHN"
		2	"2026.1"
		3	0.58
		4	"IEA/Ember 2023 grid intensity"
	9	1	"NLD"
		2	"2026.1"
		3	0.328
		4	"IEA/Ember 2023 grid intensity"
	more		
	13	1	"POL"

		2	"2026.1"
		3	0.662
		4	"IEA/Ember 2023 grid intensity"
note	"DEU 2025.1 wins; the 2026.1 GLOBAL default follows"		

```
md"""
### Dimensional fit is part of factor selection

`compatibility(f, activity_unit)` refuses to pair a factor with an activity of the
wrong dimension – so a `kgCO₂e/kWh` factor never meets a `GJ` activity.
"""
```

Dimensional fit is part of factor selection

compatibility(f, activity_unit) refuses to pair a factor with an activity of the wrong dimension — so a kgCO₂e/kWh factor never meets a GJ activity.

```
begin
    f = lookup_factor(lib, "natural_gas", "natural_gas"; region="GLOBAL")[1]
    describe_factor(f), compatibility(f, "GJ"), compatibility(f, "MWh"), compatibility(f, "t")
end
```

1	"natural_gas = 56.1 kgCO₂/GJ [CarbonAccounting built-in 2026.1, GLOBAL, DQ2, u_r=3%, IPCC 2006 Vol.2 Tab.2.2]"
2	true
3	true
4	false

```
md"""
### Resolution: free-text activity → factor

`resolve_factor` walks the rule: exact item, then the rule's default item, then an
EEIO sector hint from the procurement text, and finally the mobile-combustion set.
The same entry point serves Scope 1, 2 and 3.
"""
```

Resolution: free-text activity → factor

resolve_factor walks the rule: exact item, then the rule's default item, then an EEIO sector hint from the procurement text, and finally the mobile-combustion set. The same entry point serves Scope 1, 2 and 3.

```
begin
    rules = ["natural_gas", "refrigerant", "ldar", "flaring", "process_cement",
             "electricity", "waste", "upstream_transport"]
    items = Dict("refrigerant" => "R410A", "ldar" => nothing, "flaring" => nothing,
                 "process_cement" => "clinker", "electricity" => nothing,
                 "waste" => "recycling", "upstream_transport" => "sea_container",
                 "natural_gas" => nothing)
    DataFrame(source_type=rules,
              chosen_factor=[resolve_factor(lib, SOURCE_RULES[r];
                                             item=get(items, r, nothing), region="GLOBAL",
                                             date=Date(2024, 12, 31)) |> f -> f === nothing ? "-" : f.ef_id
                             for r in rules])
end
```

1	"natural_gas"	"mobile_combustion_diesel_fleet_CO2e_GLOBAL_2026.1"
2	"refrigerant"	"refrigerant_R410A_CO2e_GLOBAL_2026.1"
3	"ldar"	"ldar_component_leak_CH4_GLOBAL_2026.1"
4	"flaring"	"flaring_flare_gas_CO2e_GLOBAL_2026.1"
5	"process_cement"	"_"
6	"electricity"	"electricity_grid_average_CO2e_GLOBAL_2026.1"
7	"waste"	"waste_recycling_CO2e_GLOBAL_2026.1"
8	"upstream_transport"	"upstream_transport_sea_container_CO2e_GLOBAL_2026.1"

```
md"""
### Spend-based factors: from purchase-order text to an EEIO sector

`purchase → sector_hint → EF_EEIO`, with the spend deflated to the base year of the
factor set.
"""
```

Spend-based factors: from purchase-order text to an EEIO sector

purchase → sector_hint → EF_EEIO, with the spend deflated to the base year of the factor set.

```
begin
    texts = ["stainless steel plate", "facility management", "corrugated packaging",
             "cold-chain logistics", "agricultural fertiliser"]
    idx = PriceIndex(2024, Dict(2020 => 100.0, 2021 => 104.7, 2022 => 113.2, 2023 => 117.9,
                                2024 => 121.9))
    DataFrame(text=texts, sector=sector_hint.(texts),
              spend_2023_deflated=[round(deflate(1_000_000, 2023, idx); digits=0) for _ in texts])
end
```

1	"stainless steel plate"	"steel"	1.03393e6
2	"facility management"	"services"	1.03393e6
3	"corrugated packaging"	"services"	1.03393e6
4	"cold-chain logistics"	"transport_services"	1.03393e6
5	"agricultural fertiliser"	"agriculture"	1.03393e6

```
md"""
### Every factor carries its own uncertainty and quality

That is what lets the uncertainty engine (notebook 10) propagate a *defensible*
budget instead of a blanket percentage.
"""
```

Every factor carries its own uncertainty and quality

That is what lets the uncertainty engine (notebook 10) propagate a *defensible* budget instead of a blanket percentage.

```
combine(groupby(factors_dataframe(lib), :category),
         :uncertainty => (x -> round(100 * mean(x); digits=1)) => :mean_u_r_pct, nrow => :records)
```

1	"natural_gas"	34.3	3
2	"fuel_oil"	34.3	3
3	"diesel_stationary"	34.0	3
4	"lpg"	34.3	3
5	"coal"	21.0	3
6	"biomass"	35.0	3
7	"blast_furnace_gas"	35.0	3
8	"mobile_combustion"	3.6	5
9	"electricity"	10.3	20
10	"ppa_electricity"	21.7	3
	more		
39	"investments"	45.0	17

```
begin
    out = joinpath(CarbonAccounting.builddir("notebook03"), "factors_snapshot.csv")
    export_factors(lib, out)
    reloaded = load_factors(out; name="snapshot")
    println("exported ", length(lib), " factors to ", out)
    println("reloaded ", length(reloaded), " factors, version ", reloaded.version)
    println("Notebook 03 - the factor set is versioned, provenance-documented, dimension-checked and
exportable.")
end
```

04_Scope1_Direct_Emissions

17 cells · executed in 19.94 s

```
begin
    using CarbonAccounting, CSV, DataFrames, Dates, Statistics, Printf
    using Plots
    default(fmt=:svg, legend=:topleft, size=(760, 420), dpi=120)
    datadir(parts...) = CarbonAccounting.datadir(parts...)
end
```

datadir (generic function with 1 method)

```
md"""
# 04 · Scope 1 – direct emissions

Five formulas, each with its own physics, all returning `EmissionRow`s:


$$E_{\text{stat}} = \sum_i \dot{Q}_i / \eta_{\text{b}} \cdot (EF_{\text{CO}_2, i} \cdot OF_i + GWP_{\text{CH}_4} \cdot EF_{\text{CH}_4, i} + GWP_{\text{N}_2\text{O}} \cdot EF_{\text{N}_2\text{O}, i})$$


$$E_{\text{flare}} = \dot{V} \cdot EF \cdot DRE \cdot \eta_{\text{comb}} + GWP_{\text{CH}_4} \cdot \dot{V} \cdot \rho_{\text{CH}_4} \cdot x_{\text{CH}_4} \cdot (1 - DRE \cdot \eta_{\text{comb}})$$


$$E_{\text{fug}} = (C_{\text{start}} + C_{\text{purchased}} - C_{\text{end}} - C_{\text{recovered}} - C_{\text{disposed}}) \cdot GWP_{\text{CO}_2}$$


$$E_{\text{LDAR}} = N \cdot \text{leak\_rate} \cdot GWP_{\text{CH}_4}$$


$$E_{\text{proc}} = P \cdot EF \quad (\text{stoichiometry per tonne of product})$$


The engines never return a bare number: each row carries its factor id, version,
share @i, data-quality class and uncertainty.
"""
```

04 · Scope 1 — direct emissions

Five formulas, each with its own physics, all returning EmissionRows:

```
E_stat =  $\sum_i \dot{Q}_i / \eta_{\text{b}} \cdot (EF_{\text{CO}_2, i} \cdot OF_i + GWP_{\text{CH}_4} \cdot EF_{\text{CH}_4, i} + GWP_{\text{N}_2\text{O}} \cdot EF_{\text{N}_2\text{O}, i})$ 
E_flare =  $\dot{V} \cdot EF \cdot DRE \cdot \eta_{\text{comb}} + GWP_{\text{CH}_4} \cdot \dot{V} \cdot \rho_{\text{CH}_4} \cdot x_{\text{CH}_4} \cdot (1 - DRE \cdot \eta_{\text{comb}})$ 
E_fug =  $(C_{\text{start}} + C_{\text{purchased}} - C_{\text{end}} - C_{\text{recovered}} - C_{\text{disposed}}) \cdot GWP_{\text{CO}_2}$ 
E_LDAR =  $N \cdot \text{leak\_rate} \cdot GWP_{\text{CH}_4}$ 
E_proc =  $P \cdot EF$  (stoichiometry per tonne of product)
```

The engines never return a bare number: each row carries its factor id, version, share ©_i, data-quality class and uncertainty.

```
md"""### Stationary combustion, gas by gas – oxidation factor and boiler efficiency"
```

Stationary combustion, gas by gas — oxidation factor and boiler efficiency

```
begin
     $\dot{Q}_i = 1.0e5$  # GJ of fuel input
    rows = [(fuel, combustion_emissions( $\dot{Q}_i$ ; EF_CO2=56.1, EF_CH4=1e-3, EF_N2O=1e-4,
                                           OF=1.0,  $\eta_{\text{b}}$ =0.9, fossil=(fuel != "biomass"))
            for fuel in ["natural_gas", "fuel_oil", "coal"]]
    DataFrame(fuel=[r[1] for r in rows], E=[r[2].E for r in rows],
              E_CO2=[r[2].E_CO2 for r in rows], E_CH4=[r[2].E_CH4 for r in rows],
              E_N2O=[r[2].E_N2O for r in rows])
end
```

1	"natural_gas"	6239.68	6233.33	0.111111	0.0111111
2	"fuel_oil"	6239.68	6233.33	0.111111	0.0111111
3	"coal"	6239.68	6233.33	0.111111	0.0111111

```
md"""
### Flaring is not one multiplication

Only the fraction  $\phi = \text{DRE} \cdot \eta_{\text{comb}}$  of the gas is destroyed; the rest leaves as
**methane**, which at  $\text{GWP}_{100} = 29.8$  dominates the result.
"""
```

Flaring is not one multiplication

Only the fraction $\phi = \text{DRE} \cdot \eta_{\text{comb}}$ of the gas is destroyed; the rest leaves as **methane**, which at $\text{GWP}_{100} = 29.8$ dominates the result.

```
begin
    V̇ = 1.0e6 # m³(n) of flare gas in the year
    r = flare_emissions(V̇; EF_flare=1.85, DRE=0.98, η_comb=0.995, x_CH₄=0.8)
    (E=r.E, E_combusted=r.E_combusted, E_slip=r.E_slip, φ=r.φ,
     m_CH₄_slip_kg=r.m_CH₄_slip, note="slip is $(round(100*r.E_slip/r.E; digits=1)) % of the total")
end
```

E	2229.44
E_combusted	1803.93
E_slip	425.504
φ	0.9751
m_CH₄_slip_kg	14278.7
note	"slip is 19.1 % of the total"

```
md"""### Fugitive: refrigerant mass balance and the LDAR component count
```

Fugitive: refrigerant mass balance and the LDAR component count

```
begin
    ref = fugitive_refrigerant(; C_start=1_200, C_purchased=140, C_end=1_305,
                               C_recovered=25, C_disposed=0, GWP₁₀₀=2088) # R410A
    ldar = fugitive_ldar(1_850; leak_rate=0.5)
    (refrigerant=ref, ldar=ldar,
     note="the mass balance finds $(ref.m_lost) kg lost; the LDAR estimate is an independent view")
end
```

refrigerant	E	20.88
	m_lost	10
ldar	E	27.565
	m_CH₄	925.0
note	"the mass balance finds 10 kg lost; the LDAR estimate is an independent view"	

```
md"""### Process emissions: stoichiometry per tonne of product"
```

Process emissions: stoichiometry per tonne of product

```
begin
    cases = [("clinker", 42_000.0, 0.525, "CO2", "tCO₂/t"),
              ("lime", 12_000.0, 0.785, "CO2", "tCO₂/t"),
              ("nitric_acid", 3_100.0, 5.7, "N2O", "kgN₂O/t"),
              ("ammonia", 8_000.0, 1.90, "CO2e", "tCO₂e/t")]
    DataFrame(product=[c[1] for c in cases], production_t=[c[2] for c in cases],
              EF=[c[3] for c in cases], unit=[c[5] for c in cases],
              E_tCO₂e=[round(process_emissions(c[2]; EF=c[3], gas=c[4], unit=c[5]).E; digits=1) for c in
cases])
end
```

1	"clinker"	42000.0	0.525	"tCO₂/t"	22050.0
2	"lime"	12000.0	0.785	"tCO₂/t"	9420.0
3	"nitric_acid"	3100.0	5.7	"kgN₂O/t"	4823.9
4	"ammonia"	8000.0	1.9	"tCO₂e/t"	15200.0

```
md"""
### The portfolio

`compute_scope1` routes every activity record to the right formula of the table
above (see the docstring of `scope1_row`), applies the consolidation share @i and
keeps `E_stat`, `E_mob`, `E_fug`, `E_flare`, `E_proc` apart.
"""
```

The portfolio

compute_scope1 routes every activity record to the right formula of the table above (see the docstring of scope1_row), applies the consolidation share ©_i and keeps E_stat, E_mob, E_fug, E_flare, E_proc apart.

```
begin
    records = ingest_files([datadir("raw", "activity_erp_2024.csv")];
                        entity_default="AcmeIndustrial_SA").records
    scope1 = compute_scope1(records; consolidation=:operational, date=Date(2024, 12, 31))
    scope1
end
```

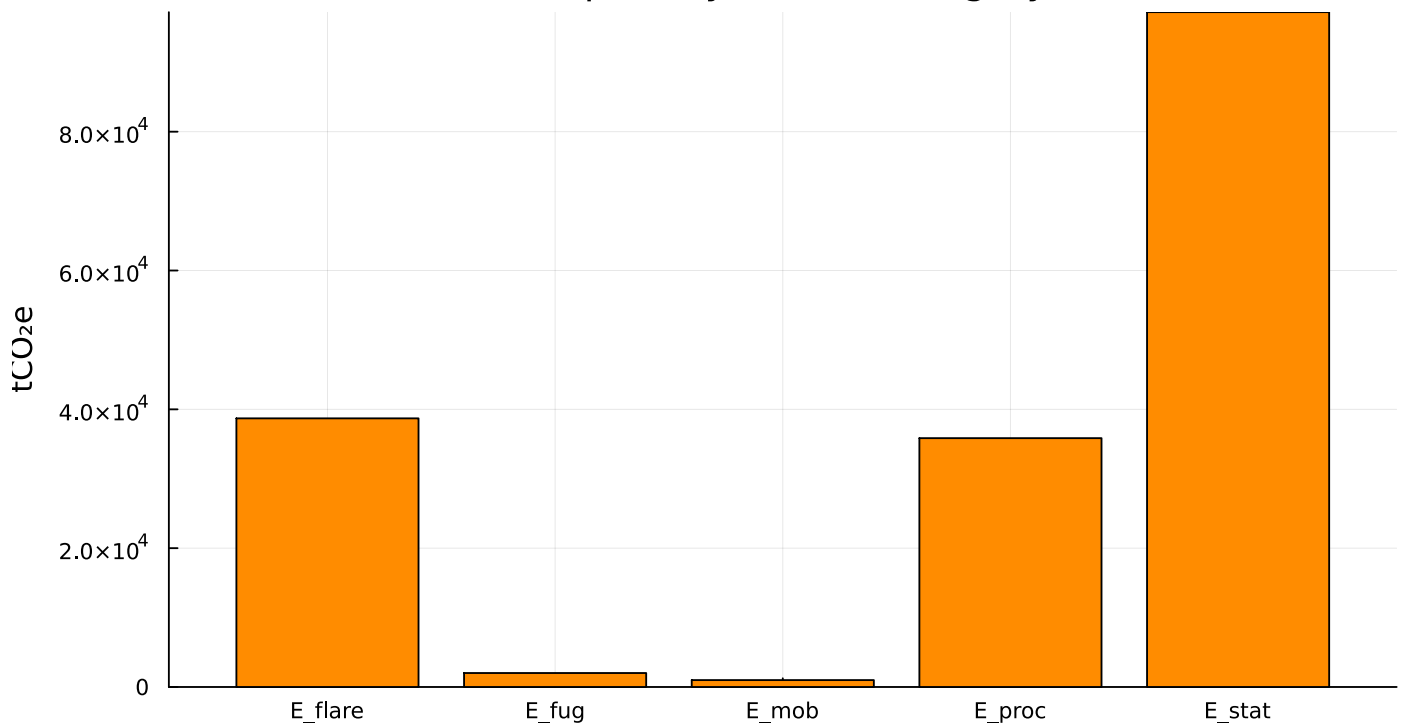
rows	1	activity_erp_2024.csv:2 natural_gas Erdgas Kessel 1 30184.000 GJ IPCC tier 1 (gas-specific, OF, η_b) 1883.3843 tCO ₂ e DQ2						
	2	activity_erp_2024.csv:3 diesel_fleet Diesel Gerador 5.595 m ³ average factor (CO ₂ e) 15.1054 tCO ₂ e DQ2						
	3	activity_erp_2024.csv:4 natural_gas Erdgas Kessel 1 27259.000 GJ IPCC tier 1 (gas-specific, OF, η_b) 1700.8738 tCO ₂ e DQ2						
	4	activity_erp_2024.csv:5 diesel_fleet Diesel Gerador 6.416 m ³ average factor (CO ₂ e) 17.3219 tCO ₂ e DQ2						
	5	activity_erp_2024.csv:6 natural_gas Erdgas Kessel 1 27010.000 GJ IPCC tier 1 (gas-specific, OF, η_b) 1685.3370 tCO ₂ e DQ2						
	6	activity_erp_2024.csv:7 diesel_fleet Diesel Gerador 5.481 m ³ average factor (CO ₂ e) 14.7976 tCO ₂ e DQ2						
	7	activity_erp_2024.csv:8 natural_gas Erdgas Kessel 1 22205.000 GJ IPCC tier 1 (gas-specific, OF, η_b) 1385.5205 tCO ₂ e DQ2						
	8	activity_erp_2024.csv:9 diesel_fleet Diesel Gerador 5.874 m ³ average factor (CO ₂ e) 15.8586 tCO ₂ e DQ2						
	9	activity_erp_2024.csv:10 natural_gas Erdgas Kessel 1 18750.000 GJ IPCC tier 1 (gas-specific, OF, η_b) 1169.9396 tCO ₂ e DQ2						
		more						
	175	activity_erp_2024.csv:177 natural_gas Erdgas Kessel 1 -8.000 GJ IPCC tier 1 (gas-specific, OF, η_b) -0.4992 tCO ₂ e DQ2						
by_source	("E_mob", MIME type text/plain)		993.904					
	("E_fug", MIME type text/plain)		2011.78					
	("E_stat", MIME type text/plain)		97225.2					
	("E_proc", MIME type text/plain)		35849.4					
	("E_flare", MIME type text/plain)		38701.1					
E_Scope1	174781.39337479963							
memo_biogenic	0.0							

```
scope1_detail(scope1) |> df -> df[1:min(8, nrow(df)), :]
```

1	"activity_erp_2024.csv:2"	"SAO-01"	"natural_gas"	"E_stat"	"Erdgas Kessel 1"	"IPCC tier 1 (gas-specific, OF, η _b)"	30184.0	"GJ"	more
2	"activity_erp_2024.csv:3"	"SAO-01"	"diesel_fleet"	"E_mob"	"Diesel Gerador"	"average factor (CO ₂ e)"	5.595	"m ³ "	more
3	"activity_erp_2024.csv:4"	"SAO-01"	"natural_gas"	"E_stat"	"Erdgas Kessel 1"	"IPCC tier 1 (gas-specific, OF, η _b)"	27259.0	"GJ"	more
4	"activity_erp_2024.csv:5"	"SAO-01"	"diesel_fleet"	"E_mob"	"Diesel Gerador"	"average factor (CO ₂ e)"	6.416	"m ³ "	more
5	"activity_erp_2024.csv:6"	"SAO-01"	"natural_gas"	"E_stat"	"Erdgas Kessel 1"	"IPCC tier 1 (gas-specific, OF, η _b)"	27010.0	"GJ"	more
6	"activity_erp_2024.csv:7"	"SAO-01"	"diesel_fleet"	"E_mob"	"Diesel Gerador"	"average factor (CO ₂ e)"	5.481	"m ³ "	more
7	"activity_erp_2024.csv:8"	"SAO-01"	"natural_gas"	"E_stat"	"Erdgas Kessel 1"	"IPCC tier 1 (gas-specific, OF, η _b)"	22205.0	"GJ"	more
8	"activity_erp_2024.csv:9"	"SAO-01"	"diesel_fleet"	"E_mob"	"Diesel Gerador"	"average factor (CO ₂ e)"	5.874	"m ³ "	more

```
begin
  parts = sort(collect(scope1.by_source))
  plt = bar([p[1] for p in parts], [p[2] for p in parts], legend=false,
            title="Scope 1 by source category", ylabel="tCO2e", color=:darkorange)
  save_figure(plt, "04_scope1_breakdown"; saver=savefig)
  plt
end
```

Scope 1 by source category



```
begin
  # data quality and uncertainty of each contributing sub-total
  df = emissions_dataframe(scope1.rows)
  sort(combine(groupby(df, :source_type),
                :E => sum => :E, :DQ => (x -> round(mean(x); digits=2)) => :mean_DQ,
                :u_rel => (x -> round(mean(x); digits=3)) => :mean_u_r), :E, rev=true)
end
```

1	"natural_gas"	97225.2	2.0	0.708
2	"flaring"	38701.1	2.0	0.254
3	"process_nitric_acid"	34892.4	2.0	0.402
4	"refrigerant"	1812.38	2.0	0.109
5	"diesel_fleet"	993.904	2.0	0.052
6	"process_cement"	956.923	2.0	0.058
7	"ldar"	199.392	4.0	0.636

```
begin
    println("E_Scope1      : ", round(scope1.E_Scope1; digits=1), " tCO₂e")
    println("biogenic memo  : ", round(scope1.memo_biogenic; digits=1), " tCO₂e (excluded from the
total)")
    println("rows          : ", length(scope1.rows))
    println("Notebook 04 – combustion, flaring, fugitive and process emissions are computed by their own
physics, not by one generic factor.")
end
```

05_Scope2_Purchased_Energy

17 cells · executed in 26.26 s

```
begin
    using CarbonAccounting, CSV, DataFrames, Dates, Statistics, Printf
    using Plots
    default(fmt=:svg, legend=:topleft, size=(760, 420), dpi=120)
    datadir(parts...) = CarbonAccounting.datadir(parts...)
end

datadir (generic function with 1 method)

md"""
# 05 · Scope 2 – purchased electricity, steam, heat and cooling

The GHG Protocol requires the dual report:


$$E_{loc} = \sum_i (C_i \cdot EF_{grid,y})$$


$$E_{mkt} = \sum_i (C_i \cdot EF_{contract}) + C_{unclaimed} \cdot EF_{residual}$$

location based
market based

`compute_scope2` emits two rows per purchase – `location-based` and
`market-based` – with the same activity, share  $\Theta_i$  and uncertainty, so both figures
decompose identically. The market factor comes from the contractual instrument
declared on the record (PPA, green tariff), otherwise the residual mix applies;
 $\lambda_{cov}$  states how much of the consumption instruments cover at all.
"""
```

05 · Scope 2 — purchased electricity, steam, heat and cooling

The GHG Protocol requires the **dual report**:

$$E_{loc} = \sum_i (C_i \cdot EF_{grid,y}) \quad \text{location based}$$
$$E_{mkt} = \sum_i (C_i \cdot EF_{contract}) + C_{unclaimed} \cdot EF_{residual} \quad \text{market based}$$

`compute_scope2` emits *two rows per purchase* — location-based and market-based — with the same activity, share Θ_i and uncertainty, so both figures decompose identically. The market factor comes from the contractual instrument declared on the record (PPA, green tariff), otherwise the residual mix applies; λ_{cov} states how much of the consumption instruments cover at all.

```
md"""### Where the market-based factor comes from – the contractual hierarchy"
```

Where the market-based factor comes from — the contractual hierarchy

```
begin
    lib = default_library()
    ppa = resolve_factor(lib, SOURCE_RULES["ppa_electricity"]; item="ppa_renewable",
                        region="GLOBAL", date=Date(2024, 12, 31)).value
    tariff = resolve_factor(lib, SOURCE_RULES["ppa_electricity"]; item="green_tariff",
                          region="GLOBAL", date=Date(2024, 12, 31)).value
    resid = resolve_factor(lib, SOURCE_RULES["electricity"]; region="GLOBAL",
                        date=Date(2024, 12, 31), market=:market).value
    DataFrame(instrument=["PPA, renewable", "Green tariff with certificates",
                        "No instrument → residual mix"],
              factor=[ppa, tariff, resid], unit="kgCO₂e/kWh")
end
```

1	"PPA, renewable"	0.0	"kgCO₂e/kWh"
2	"Green tariff with certificates"	0.05	"kgCO₂e/kWh"
3	"No instrument → residual mix"	0.62	"kgCO₂e/kWh"

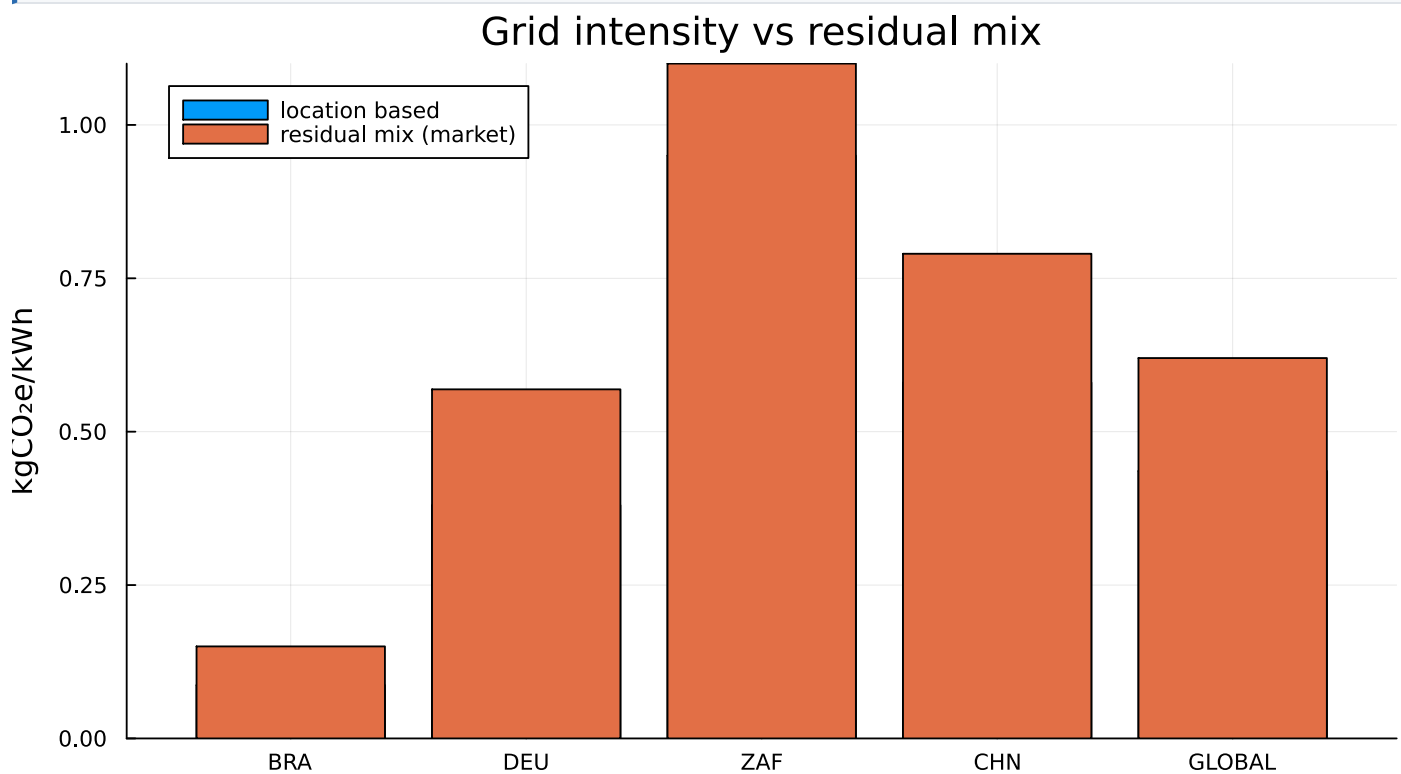
```
md"### Location-based grid intensity versus residual mix, by country"
```

Location-based grid intensity versus residual mix, by country

```
begin
  countries = ["BRA", "DEU", "ZAF", "CHN", "GLOBAL"]
  loc = [resolve_factor(lib, SOURCE_RULES["electricity"]; region=c,
                        date=Date(2024, 12, 31)).value for c in countries]
  res = [resolve_factor(lib, SOURCE_RULES["electricity"]; region=c,
                        date=Date(2024, 12, 31), market=:market).value for c in countries]
  DataFrame(country=countries, EF_grid=loc, EF_residual=res,
            premium_pct=[round(100 * (r / l - 1); digits=1) for (l, r) in zip(loc, res)])
end
```

1	"BRA"	0.087	0.15	72.4
2	"DEU"	0.38	0.569	49.7
3	"ZAF"	0.95	1.1	15.8
4	"CHN"	0.58	0.79	36.2
5	"GLOBAL"	0.436	0.62	42.2

```
begin
  plt_grid = bar(countries, [loc res], bar_position=:dodge,
                 label=["location based" "residual mix (market)"],
                 title="Grid intensity vs residual mix", ylabel="kgCO2e/kWh")
  save_figure(plt_grid, "05_grid_vs_residual"; saver=savefig)
  plt_grid
end
```



```
md"### The purchases of the portfolio – mixed units in one file, instruments per month"
```

The purchases of the portfolio — mixed units in one file, instruments per month

```
begin
    records = ingest_files([datadir("raw", "electricity_utility_2024.csv")];
                        entity_default="AcmeIndustrial_SA").records
    df = records_dataframe(records)
    (purchases=nrow(df), by_unit=combine(groupby(df, [:unit, :unit_canon]), nrow => :n),
     instruments=unique([r.meta["instrument"] for r in records]))
end
```

purchases	72			
by_unit	1	"MWh"	"GJ"	39
	2	"kWh"	"GJ"	33
instruments	1	""		
	2	"ppa_renewable"		
	3	"green_tariff"		

md"### The dual report"

The dual report

```
compute_scope2(records; date=Date(2024, 12, 31), consolidation=:operational)
```

rows	1	electricity_utility_2024.csv:2	electricity	purchased electricity	11052.000 GJ	location-based	267.0900 tCO ₂ e	DQ2
	2	electricity_utility_2024.csv:2	electricity	purchased electricity	11052.000 GJ	market-based	460.5000 tCO ₂ e	DQ2
	3	electricity_utility_2024.csv:3	electricity	purchased electricity	11786.400 GJ	location-based	284.8380 tCO ₂ e	DQ2
	4	electricity_utility_2024.csv:3	electricity	purchased electricity	11786.400 GJ	market-based	491.1000 tCO ₂ e	DQ2
	5	electricity_utility_2024.csv:4	electricity	purchased electricity	9636.566 GJ	location-based	232.8837 tCO ₂ e	DQ2
	6	electricity_utility_2024.csv:4	electricity	purchased electricity	9636.566 GJ	market-based	401.5236 tCO ₂ e	DQ2
	7	electricity_utility_2024.csv:5	electricity	purchased electricity	8963.233 GJ	location-based	216.6115 tCO ₂ e	DQ2
	8	electricity_utility_2024.csv:5	electricity	purchased electricity	8963.233 GJ	market-based	0.0000 tCO ₂ e	DQ2
	9	electricity_utility_2024.csv:6	electricity	purchased electricity	7944.642 GJ	location-based	191.9955 tCO ₂ e	DQ2
		more						
	144	electricity_utility_2024.csv:73	electricity	purchased electricity	12679.200 GJ	market-based	2782.3800 tCO ₂ e	DQ2
E_loc	73433.264921							
E_mkt	86683.22791700001							
C_elec	166869.63499999998							
λ_cov	0.19551940051885416							
by_site	1	"SAO-01"	"BRA"	1.06196e5	2566.4	1691.82		
	2	"SAO-02"	"BRA"	1.04235e5	2519.02	4343.14		
	3	"SHA-03"	"CHN"	1.18108e5	19028.5	25918.1		
	4	"HAM-01"	"DEU"	62531.5	6600.54	3032.26		
	5	"ROT-02"	"NLD"	72973.6	6648.71	9932.52		
	6	"JNB-01"	"ZAF"	1.36687e5	36070.1	41765.4		

```
begin
    scope2 = compute_scope2(records; date=Date(2024, 12, 31), consolidation=:operational)
    detail = scope2_detail(scope2)
    first(select(detail, :site, :item, :method, :Q, :unit, :ef_id, :ef_value, :ef_unit, :E, :DQ), 8)
end
```

1	"SAO-01"	"purchased electricity"	"location-based"	11052.0	"GJ"	"electricity_grid_average_CO2e_BRA_2026.1"	0.087	"kgCO ₂ e/kWh"	267.09	2
2	"SAO-01"	"purchased electricity"	"market-based"	11052.0	"GJ"	"electricity_residual_mix_CO2e_BRA_2026.1"	0.15	"kgCO ₂ e/kWh"	460.5	2
3	"SAO-01"	"purchased electricity"	"location-based"	11786.4	"GJ"	"electricity_grid_average_CO2e_BRA_2026.1"	0.087	"kgCO ₂ e/kWh"	284.838	2
4	"SAO-01"	"purchased electricity"	"market-based"	11786.4	"GJ"	"electricity_residual_mix_CO2e_BRA_2026.1"	0.15	"kgCO ₂ e/kWh"	491.1	2
5	"SAO-01"	"purchased electricity"	"location-based"	9636.57	"GJ"	"electricity_grid_average_CO2e_BRA_2026.1"	0.087	"kgCO ₂ e/kWh"	232.884	2
6	"SAO-01"	"purchased electricity"	"market-based"	9636.57	"GJ"	"electricity_residual_mix_CO2e_BRA_2026.1"	0.15	"kgCO ₂ e/kWh"	401.524	2
7	"SAO-01"	"purchased electricity"	"location-based"	8963.23	"GJ"	"electricity_grid_average_CO2e_BRA_2026.1"	0.087	"kgCO ₂ e/kWh"	216.611	2
8	"SAO-01"	"purchased electricity"	"market-based"	8963.23	"GJ"	"ppa_electricity_ppa_renewable_CO2e_GLOBAL_2026.1"	0.0	"kgCO ₂ e/kWh"	0.0	2

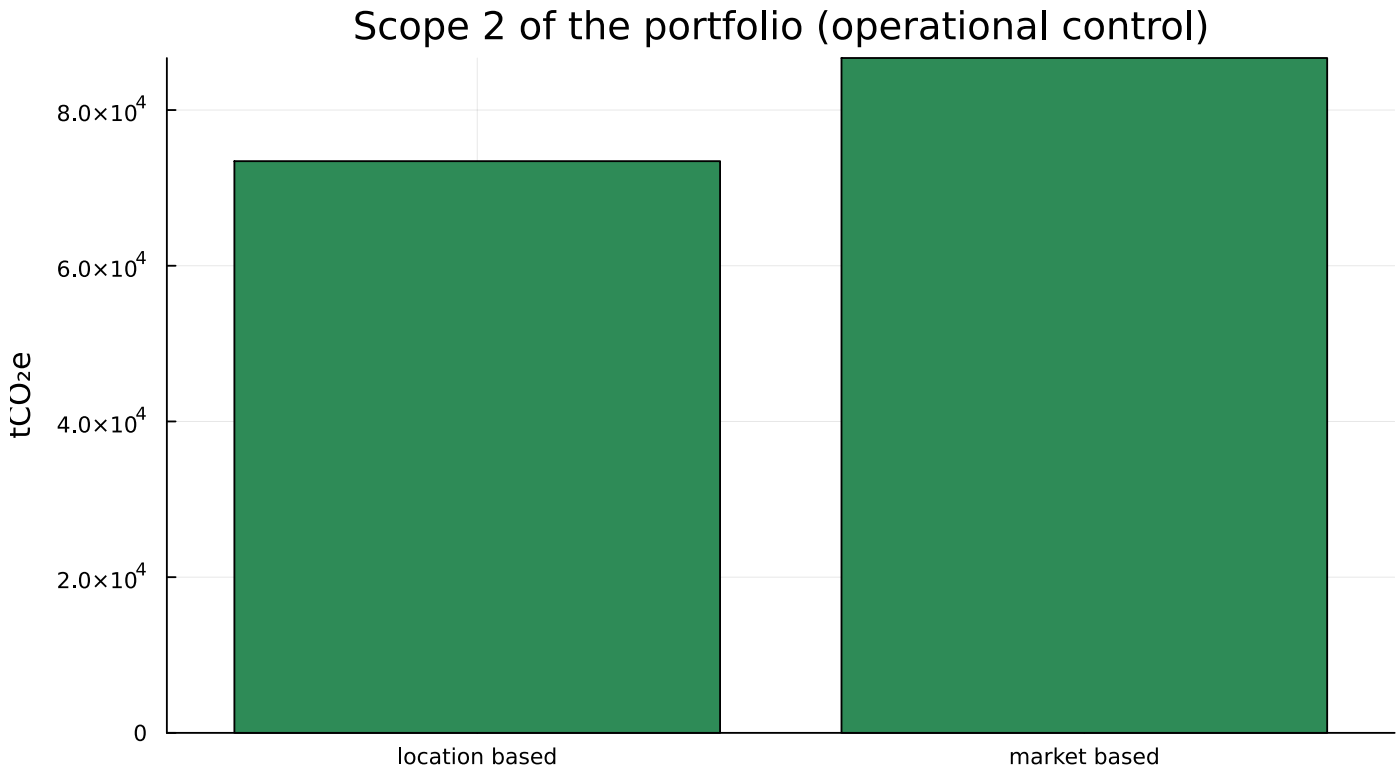
```
begin
    per_site = combine(groupby(scope2_detail(scope2), [:site, :method]),
        :Q => sum => :activity_GJ, :E => sum => :E)
    sort!(per_site, [:site, :method])
    per_site
end
```

1	"HAM-01"	"location-based"	62531.5	6600.54
2	"HAM-01"	"market-based"	62531.5	3032.26
3	"JNB-01"	"location-based"	1.36687e5	36070.1
4	"JNB-01"	"market-based"	1.36687e5	41765.4
5	"ROT-02"	"location-based"	72973.6	6648.71
6	"ROT-02"	"market-based"	72973.6	9932.52
7	"SAO-01"	"location-based"	1.06196e5	2566.4
8	"SAO-01"	"market-based"	1.06196e5	1691.82
9	"SAO-02"	"location-based"	1.04235e5	2519.02
10	"SAO-02"	"market-based"	1.04235e5	4343.14
11	"SHA-03"	"location-based"	1.18108e5	19028.5
12	"SHA-03"	"market-based"	1.18108e5	25918.1

```
md"### Dual reporting is not a formality: the two figures answer different questions"
```

Dual reporting is not a formality: the two figures answer different questions

```
begin
    plt_dual = bar(["location based", "market based"],
                    reshape([scope2.E_loc, scope2.E_mkt], 2, 1),
                    bar_position=:dodge,
                    label="", legend=false, ylabel="tCO2e",
                    title="Scope 2 of the portfolio (operational control)",
                    color=[:seagreen :darkseagreen])
    save_figure(plt_dual, "05_scope2_dual"; saver=savefig)
    plt_dual
end
```



```
begin
    println("E_Scope2 location : ", round(scope2.E_loc; digits=1), " tCO2e")
    println("E_Scope2 market   : ", round(scope2.E_mkt; digits=1), " tCO2e")
    println("purchased electricity : ", round(scope2.C_elec; digits=1), " MWh")
    println("contractual coverage λ_cov : ", round(100 * scope2.λ_cov; digits=1), " %")
    println("Notebook 05 – both Scope 2 figures come from the same activities, so their difference is attributable.")
end
```

06_Scope3_Value_Chain

18 cells · executed in 20.16 s

```
begin
    using CarbonAccounting, CSV, DataFrames, Dates, Statistics, Printf
    using Plots
    default(fmt=:svg, legend=:topleft, size=(760, 420), dpi=120)
    datadir(parts...) = CarbonAccounting.datadir(parts...)
end

datadir (generic function with 1 method)
```

```
md"""
# 06 · Scope 3 – value chain

Scope 3 is where an inventory is made or lost: 15 categories, most of them outside
the company's own meters. The engine applies the hybrid data hierarchy of the
GHG Protocol:

| method | activity | factor | used for |
|---|---|---|---|
| supplier specific | supplier-reported | supplier factor | DQ 1 |
| average data | mass, energy, t·km, p·km | industry average | DQ 2–3 |
| spend based | deflated spend | EEIO factor | DQ 4, screening |

Screening is deliberate: the Pareto hotspots carrying ~80 % of the total are exactly
the ones that must be refined with better data – and the engine names them.
"""
```

06 · Scope 3 — value chain

Scope 3 is where an inventory is made or lost: 15 categories, most of them outside the company's own meters. The engine applies the **hybrid data hierarchy** of the GHG Protocol:

method	activity	factor	used for
supplier specific	supplier-reported	supplier factor	DQ 1
average data	mass, energy, t·km, p·km	industry average	DQ 2–3
spend based	deflated spend	EEIO factor	DQ 4, screening

Screening is deliberate: the Pareto hotspots carrying ~80 % of the total are exactly the ones that must be refined with better data — and the engine names them.

```
md"""### The 15 categories and the activity each one expects"
```

The 15 categories and the activity each one expects

```
begin
    rows = [(c, SCOPE3_CATEGORY_NAMES[c], r.key, string(r.dim), r.unit_hint)
             for r in values(SOURCE_RULES) for c in [r.cat]
             if r.scope == :scope3 && c != nothing]
    sort!(DataFrame(cat=[r[1] for r in rows], category=[r[2] for r in rows],
                    source_type=[r[3] for r in rows], dimension=[r[4] for r in rows],
                    unit=[r[5] for r in rows]), :cat)
end
```

1	1	"1 Purchased goods and services"	"purchased_goods_spend"	"currency"	"USD2024"
2	1	"1 Purchased goods and services"	"purchased_goods_mass"	"mass"	"t"
3	2	"2 Capital goods"	"capital_goods"	"currency"	"USD2024"
4	3	"3 Fuel- and energy-related activities"	"fuel_energy_upstream"	"energy"	"GJ"
5	4	"4 Upstream transportation and distribution"	"upstream_transport"	"transport_work"	"t-km"
6	5	"5 Waste generated in operations"	"waste"	"mass"	"t"
7	6	"6 Business travel"	"business_travel_air"	"transport_work"	"p-km"
8	6	"6 Business travel"	"business_travel_land"	"transport_work"	"p-km"
9	7	"7 Employee commuting"	"employee_commuting"	"transport_work"	"p-km"
10	8	"8 Upstream leased assets"	"upstream_leased"	"currency"	"USD2024"
	more				
17	15	"15 Investments"	"investments"	"currency"	"USD2024"

```
md"""
### Spend before deflation is not spend after it

An EEIO factor is expressed per unit of spend in its own base year, so a 2023
invoice is deflated to USD2024 before it meets the factor:
`s_spend = amount · CPI(base) / CPI(year)`
"""
```

Spend before deflation is not spend after it

An EEIO factor is expressed per unit of spend *in its own base year*, so a 2023 invoice is deflated to USD2024 before it meets the factor: $s_spend = amount \cdot CPI(base) / CPI(year)$.

```
begin
    cpi = CSV.read(datadir("reference", "price_index.csv"), DataFrame)
    idx = PriceIndex(2024, Dict{Int{r.year} => Float64{r.cpi_usd} for r in eachrow(cpi)})
    DataFrame(year=cpi.year, cpi_usd=cpi.cpi_usd,
              in_USD2024_of_1M=[round(deflate(1.0e6, Int{r.year}, idx); digits=0) for r in eachrow(cpi)])
end
```

1	2020	100.0	1.219e6
2	2021	104.7	1.16428e6
3	2022	113.2	1.07686e6
4	2023	117.9	1.03393e6
5	2024	121.9	1.0e6
6	2025	125.0	975200.0
7	2026	128.2	950858.0

```
md"""### Two physical methods side by side: distance-based freight and waste treatment"
```

Two physical methods side by side: distance-based freight and waste treatment

```
begin
    lib = default_library()
    hgv = resolve_factor(lib, SOURCE_RULES["upstream_transport"]; item="hgv_diesel",
                        date=Date(2024, 12, 31))
    waste = resolve_factor(lib, SOURCE_RULES["waste"]; item="landfill_msw",
                        date=Date(2024, 12, 31))
    (Freight_E=emission(q"4.0e6 t·km", Quantity(hgv.value, hgv.unit)).val,
     freight_factor=describe_factor(hgv),
     Landfill_E=emission(q"620 t", Quantity(waste.value, waste.unit)).val,
     waste_factor=describe_factor(waste))
end
```

Freight_E	428.0
freight_factor	"hgv_diesel = 0.107 kgCO ₂ e/t·km [CarbonAccounting built-in 2026.1, GLOBAL, DQ3, u_r=15%, DEFRA 2023 HGV (average laden)]"
Landfill_E	289.54
waste_factor	"landfill_msw = 0.467 kgCO ₂ e/kg [CarbonAccounting built-in 2026.1, GLOBAL, DQ3, u_r=35%, US EPA WARM / DEFRA 2023 landfill]"

```
md"### From purchase-order text to an EEIO factor"
```

From purchase-order text to an EEIO factor

```
begin
    po = ["stainless steel plate (2 t)", "facility management Q3", "corrugated packaging",
          "cold-chain logistics", "IT consulting"]
    spend = [420_000.0, 96_000.0, 72_000.0, 210_000.0, 140_000.0]
    ef(s) = resolve_factor(lib, SOURCE_RULES["purchased_goods_spend"]; item=s,
                        date=Date(2024, 12, 31)).value
    DataFrame(purchase=po, sector=sector_hint(po), spend_USD2024=spend,
              EF_kg_per_USD=[ef(sector_hint(p)) for p in po],
              E_tCO₂e=[emission(Quantity(s, "USD2024"), Quantity(ef(sector_hint(p)),
                          "kgCO₂e/USD2024")).val for (p, s) in zip(po, spend)])
end
```

1	"stainless steel plate (2 t)"	"steel"	420000.0	0.85	357.0
2	"facility management Q3"	"services"	96000.0	0.035	3.36
3	"corrugated packaging"	"services"	72000.0	0.035	2.52
4	"cold-chain logistics"	"transport_services"	210000.0	0.35	73.5
5	"IT consulting"	"services"	140000.0	0.035	4.9

```
md"### The value chain of the portfolio"
```

The value chain of the portfolio

```

begin
    files = ["procurement_2024.csv", "logistics_travel_2024.csv", "waste_2024.csv",
            "activity_erp_2024.csv", "electricity_utility_2024.csv"]
    records = ingest_files([datadir("raw", f) for f in files];
                          entity_default="AcmeIndustrial_SA").records
    scope3 = compute_scope3(records; date=Date(2024, 12, 31), consolidation=:operational,
                          hotspot_share=0.8)

    scope3
end

```

rows	1	procurement_2024.csv:2 purchased_goods_spend corrugated packaging 21119.600 USD2024 spend based (EEIO, deflated) 0.7392 tCO ₂ e DQ4					
	2	procurement_2024.csv:3 purchased_goods_spend IT consulting 206536.000 USD2024 spend based (EEIO, deflated) 7.2288 tCO ₂ e DQ4					
	3	procurement_2024.csv:4 purchased_goods_spend facility management 55172.700 USD2024 spend based (EEIO, deflated) 1.9310 tCO ₂ e DQ4					
	4	procurement_2024.csv:5 purchased_goods_spend stainless steel plate 1033649.100 USD2024 spend based (EEIO, deflated) 878.6017 tCO ₂ e DQ4					
	5	procurement_2024.csv:6 purchased_goods_spend industrial chemicals 38589.460 USD2024 spend based (EEIO, deflated) 16.2076 tCO ₂ e DQ4					
	6	procurement_2024.csv:7 purchased_goods_spend cement bulk 132752.000 USD2024 spend based (EEIO, deflated) 86.2888 tCO ₂ e DQ4					
	7	procurement_2024.csv:8 purchased_goods_spend cement bulk 212188.900 USD2024 spend based (EEIO, deflated) 137.9228 tCO ₂ e DQ4					
	8	procurement_2024.csv:9 purchased_goods_spend polymer resin 86315.600 USD2024 spend based (EEIO, deflated) 47.4736 tCO ₂ e DQ4					
	9	procurement_2024.csv:10 purchased_goods_spend industrial chemicals 43325.940 USD2024 spend based (EEIO, deflated) 18.1969 tCO ₂ e DQ4					
		more					
	126	waste_2024.csv:19 waste incineration 268.000 t average data (industry) 5.6280 tCO ₂ e DQ3					

by_category	1	1	"1 Purchased goods and services"	4162.14	0.332713	42	4.0	0.474342
	2	4	"4 Upstream transportation and distribution"	1349.18	0.107851	18	3.0	0.206128
	3	5	"5 Waste generated in operations"	1724.34	0.13784	18	3.0	0.359212
	4	6	"6 Business travel"	582.3	0.0465479	18	3.0	0.191574
	5	7	"7 Employee commuting"	4089.19	0.326882	18	3.0	0.23065
	6	9	"9 Downstream transportation and distribution"	602.539	0.0481658	12	3.0	0.203151

E_Scope3	12509.686021500002							
hotspots	1	"1 Purchased goods and services"						
	2	"7 Employee commuting"						
	3	"5 Waste generated in operations"						
	4	"4 Upstream transportation and distribution"						

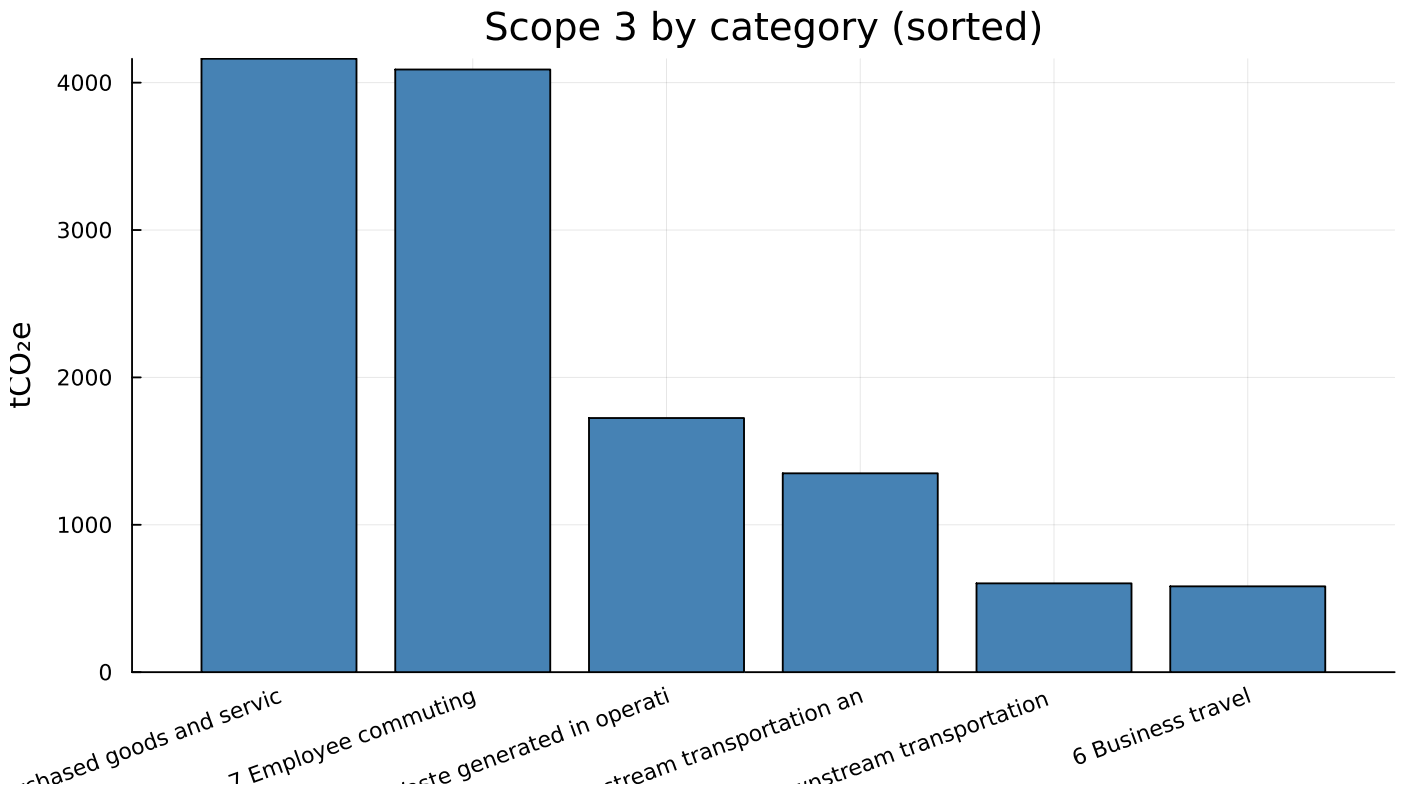
```

begin
    bycat = sort(scope3.by_category, :E, rev=true)
    bycat.category = [first(c, 28) for c in bycat.category]
    bycat
end

```

1	1	"1 Purchased goods and servic"	4162.14	0.332713	42	4.0	0.474342
2	7	"7 Employee commuting"	4089.19	0.326882	18	3.0	0.23065
3	5	"5 Waste generated in operati"	1724.34	0.13784	18	3.0	0.359212
4	4	"4 Upstream transportation an"	1349.18	0.107851	18	3.0	0.206128
5	9	"9 Downstream transportation "	602.539	0.0481658	12	3.0	0.203151
6	6	"6 Business travel"	582.3	0.0465479	18	3.0	0.191574

```
begin
    plt_cat = bar(bycat.category, bycat.E, legend=false, xrotation=20, ylabel="tCO2e",
                  title="Scope 3 by category (sorted)", color=:steelblue)
    save_figure(plt_cat, "06_scope3_categories"; saver=savefig)
    plt_cat
end
```



```
md"""
### Hotspots and the method mix

The hotspots are the categories that together carry `hotspot_share` of the total –
the refinement list. The method mix shows how much of the Scope 3 figure rests on
spend-based proxies, i.e. on estimates that must be replaced before assurance.
"""
```

Hotspots and the method mix

The hotspots are the categories that together carry `hotspot_share` of the total — the refinement list. The method mix shows how much of the Scope 3 figure rests on spend-based proxies, i.e. on estimates that must be replaced before assurance.

```
begin
    mix = combine(groupby(emissions_dataframe(scope3.rows), :method),
                  :E => sum => :E, nrow => :rows)
    mix.share = mix.E ./ sum(mix.E)
    (hotspots=scope3.hotspots, method_mix=mix)
end
```

hotspots	1	"1 Purchased goods and services"			
	2	"7 Employee commuting"			
	3	"5 Waste generated in operations"			
	4	"4 Upstream transportation and distribution"			
method_mix	1	"spend based (EEIO, deflated)"	4162.14	42	0.332713
	2	"average data (industry)"	4258.36	66	0.340405
	3	"proxy estimate"	4089.19	18	0.326882

```
begin
    println("E_Scope3          : ", round(scope3.E_Scope3; digits=1), " tCO2e")
    println("categories used : ", nrow(scope3.by_category), " of 15")
    println("hotspots         : ", join(scope3.hotspots, " · "))
    println("Notebook 06 – every category carries a method and a data-quality class, and the hotspots are
named rather than assumed.")
end
```

07_SoftSensor_Neural_Net

20 cells · executed in 22.26 s

```
begin
    using CarbonAccounting, CSV, DataFrames, Dates, Statistics, Printf
    using Plots
    default(fmt=:svg, legend=:topright, size=(760, 420), dpi=120)
    datadir(parts...) = CarbonAccounting.datadir(parts...)
    const TAGS = ["load_pct", "stack_temp_C", "o2_pct", "flare_flow_m3h", "pressure_bar",
                  "feedstock_tph"]
    const p_CH₄ = CarbonAccounting.p_CH₄          # kg/m³ at normal conditions
end
```

0.7168

```
md"""
# 07 · A neural soft sensor for the annual analyzer campaign

The operational reality: the expensive measurement – a mobile analyzer, a lab
campaign, a certified stack test – happens **once a year for one week**,
`N_camp = 7 d · 24 h = 168` samples, while the plant produces continuous data every
hour. The soft sensor learns the mapping

     $\hat{y} = f_{\theta}(x)$        $x$  : continuous DCS/SCADA tags (load, temperature, O₂, flow, ...)
                                 $y$  : what the campaign measures (here: CH₄ mole fraction %)

and then predicts `y` for all 8 760 hours of the year – with a quantified
uncertainty instead of a smoothed annual average.
"""
```

07 · A neural soft sensor for the annual analyzer campaign

The operational reality: the expensive measurement — a mobile analyzer, a lab campaign, a certified stack test — happens **once a year for one week**, $N_{\text{camp}} = 7 \text{ d} \cdot 24 \text{ h} = 168$ samples, while the plant produces continuous data every hour. The soft sensor learns the mapping

$\hat{y} = f_{\theta}(x)$ x : continuous DCS/SCADA tags (load, temperature, O₂, flow, ...)
 y : what the campaign measures (here: CH₄ mole fraction %)

and then predicts y for all 8 760 hours of the year — with a quantified uncertainty instead of a smoothed annual average.

```
md"""
### The network, written as the equations


$$z^{(1)} = W^{(1)}a^{(1-1)} + b^{(1)} \quad a^{(1)} = g(z^{(1)}) \quad \hat{y} = W^{(L)}a^{(L-1)} + b^{(L)}$$


$$\delta^{(L)} = \partial L / \partial \hat{y} \quad \delta^{(1)} = (W^{(1+1)})^T \delta^{(1+1)} \odot g'(z^{(1)}) \quad \partial L / \partial W^{(1)} = \delta^{(1)} (a^{(1-1)})^T + \lambda_{L2} W^{(1)}$$


Trained with mini-batch Adam ( $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ ) and early stopping on a
validation split. Two output heads make it heteroscedastic: it predicts  $\hat{y}$  and
 $\hat{\sigma}(x)$ , its own noise level, through the Gaussian NLL


$$L = \frac{1}{2}[(\mu - y)^2 e^{-s} + s], \quad s = \log \hat{\sigma}^2$$

"""
```

The network, written as the equations

$$z^{(1)} = W^{(1)}a^{(1-1)} + b^{(1)} \quad a^{(1)} = g(z^{(1)}) \quad \hat{y} = W^{(L)}a^{(L-1)} + b^{(L)}$$

$$\delta^{(L)} = \partial L / \partial \hat{y} \quad \delta^{(1)} = (W^{(1+1)})^T \delta^{(1+1)} \odot g'(z^{(1)}) \quad \partial L / \partial W^{(1)} = \delta^{(1)} (a^{(1-1)})^T + \lambda_{L2} W^{(1)}$$

Trained with mini-batch **Adam** ($\beta_1 = 0.9$, $\beta_2 = 0.999$) and early stopping on a validation split. Two output heads make it *heteroscedastic*: it predicts $\hat{y}$ and $\hat{\sigma}(x)$, its own noise level, through the Gaussian NLL

$$L = \frac{1}{2}[(\mu - y)^2 e^{-s} + s], \quad s = \log \hat{\sigma}^2$$

```
begin
    campaign = CSV.read(datadir("raw", "analyzer_campaign_2024.csv"), DataFrame)
    dcs = CSV.read(datadir("raw", "dcs_hourly_2024.csv"), DataFrame)
    X = Matrix(campaign[:, TAGS]) # campaign features
    y = Vector(campaign.ch4_fraction_pct) # what the analyzer measured
    X_year = Matrix(dcs[:, TAGS]) # continuous data for the year
    (campaign=size(X), year=size(X_year),
     tags=DataFrame(tag=TAGS,
                    campaign_min=vec(minimum(X; dims=1)), campaign_max=vec(maximum(X; dims=1)),
                    year_min=vec(minimum(X_year; dims=1)), year_max=vec(maximum(X_year; dims=1))))
end
```

campaign	1	168				
	2	6				
year	1	8760				
	2	6				
tags	1	"load_pct"	26.4015	85.1292	38.6551	104.0
	2	"stack_temp_C"	312.804	349.21	314.414	383.699
	3	"o2_pct"	2.81724	4.20234	2.09842	5.07404
	4	"flare_flow_m3h"	2949.5	5710.87	3100.57	6871.2
	5	"pressure_bar"	2.40751	2.7237	2.00612	2.86971
	6	"feedstock_tph"	11.0605	17.5904	11.4712	20.0019

```
md"""### Train, and look at the convergence – not only at the final number"
```

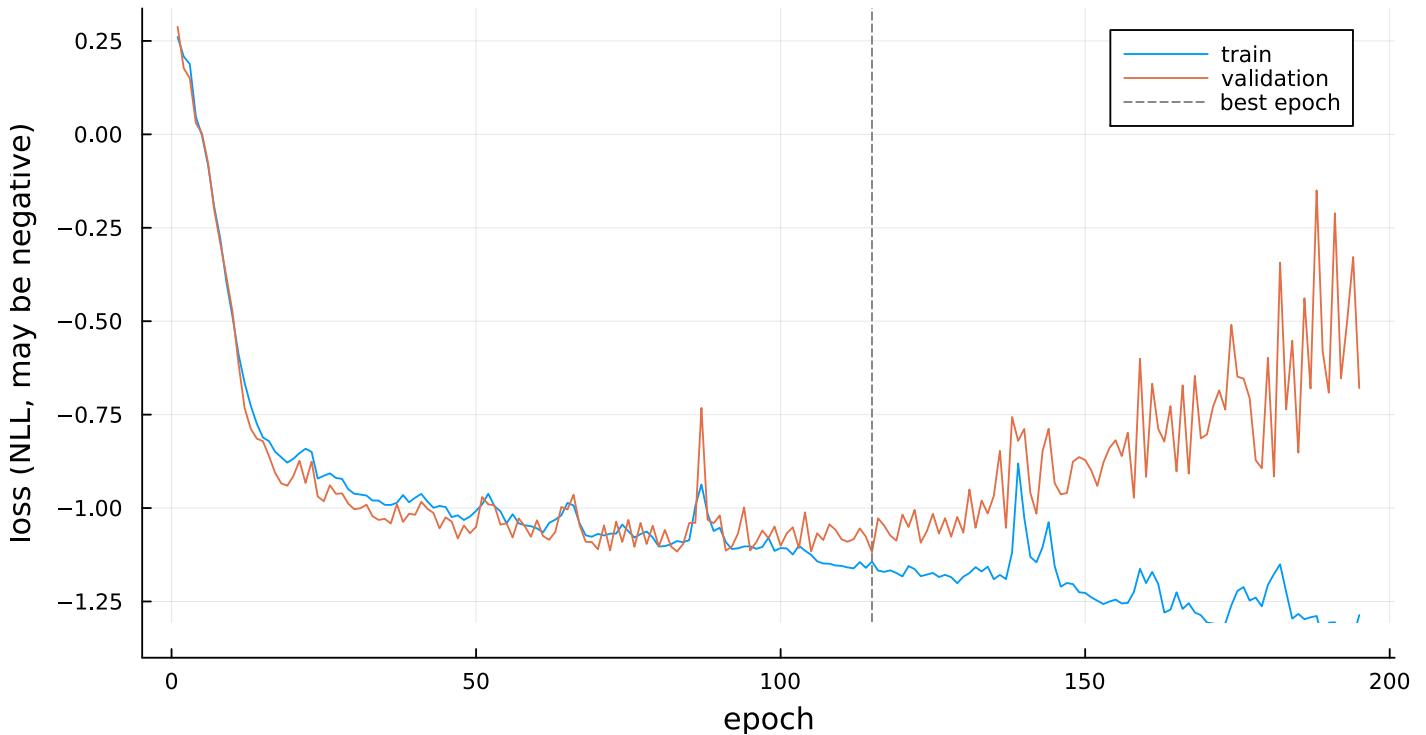
Train, and look at the convergence — not only at the final number

```
begin
    net = train_soft_sensor(X, y; hidden=[16, 8], heteroscedastic=true,
                           epochs=1200, η=0.01, λ_L2=1e-4, seed=3)
    net
end
```

```
MLP(heteroscedastic, layers 6→16×8×2, 266 θ, nll, 115 epochs)
```

```
begin
    h = training_history(net)
    plt_train = plot(h.epoch, [h.train h.val], label=["train" "validation"],
                    xlabel="epoch", ylabel="loss (NLL, may be negative)", title="Convergence (early stopping)")
    vline!(plt_train, [net.trained_epochs], label="best epoch", ls=:dash, color=:grey)
    save_figure(plt_train, "07_training_history"; saver=savefig)
    plt_train
end
```

Convergence (early stopping)



```
md"""
### Scores that matter: error *and* honesty

RMSE and  $R^2$  say how close the predictions are; `coverage95` says whether the stated
uncertainty can be trusted. A model with 95 % coverage has error bars that mean
something – the property a compliance team actually needs.
"""
```

Scores that matter: error *and* honesty

RMSE and R^2 say how close the predictions are; `coverage95` says whether the stated uncertainty can be trusted. A model with 95 % coverage has error bars that mean something — the property a compliance team actually needs.

```
begin
    ŷ = predict(net, X)
    iv = predict_interval(net, X)
    metrics_table(softsensor_metrics(y, ŷ; intervals=iv))
end
```

1	168	0.0357626	0.0274398	0.951101	0.000697688	0.97619	0.0351208
---	-----	-----------	-----------	----------	-------------	---------	-----------

```
cross_validate_softsensor(X, y; k=5, epochs=600, seed=11)
```

1	1	134	34	0.0476642	0.0376726	0.918382
2	2	134	34	0.0486917	0.0368909	0.904745
3	3	134	34	0.0429147	0.0340811	0.939403
4	4	135	33	0.0381933	0.029796	0.922544
5	5	135	33	0.0449881	0.0334754	0.925781

```
md"""
### An ensemble, so that uncertainty has two components
```

$$\sigma_{\text{total}}^2 = \sigma_{\text{epistemic}}^2 + \hat{\sigma}_{\text{aleatoric}}^2$$

```

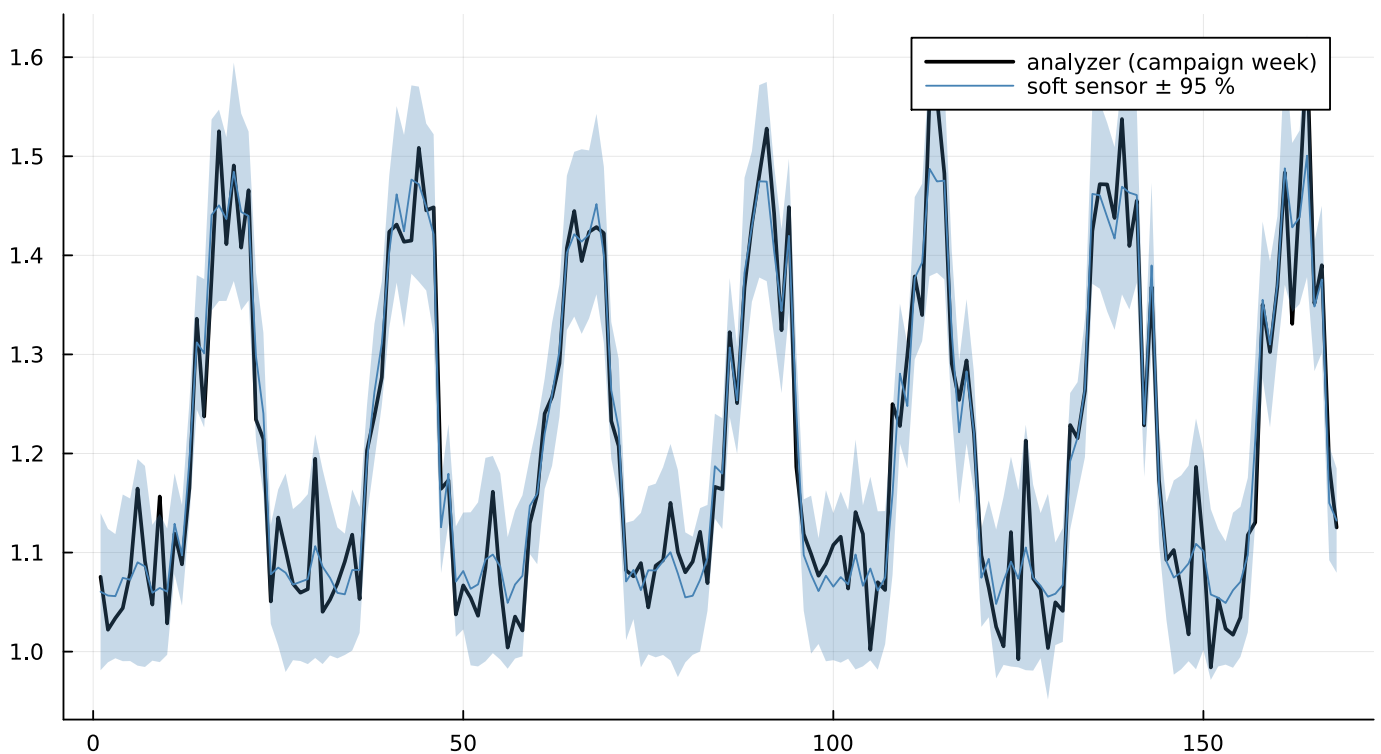
The spread between ensemble members is the epistemic part – how much the answer
depends on which week happened to be measured – while the heteroscedastic head gives
the aleatoric part per hour.
"""
```

An ensemble, so that uncertainty has two components

$$\sigma_{\text{total}}^2 = \sigma_{\text{epistemic}}^2 + \hat{\sigma}_{\text{aleatoric}}^2$$

The spread between ensemble members is the epistemic part — how much the answer depends on which week happened to be measured — while the heteroscedastic head gives the aleatoric part per hour.

```
begin
  nets = train_ensemble(X, y; n_models=5, epochs=800, heteroscedastic=true, η=0.01)
  ivc = ensemble_interval(nets, X)
  lower = [[i.ŷ - i.lo for i in ivc]...]
  upper = [[i.hi - i.ŷ for i in ivc]...]
  plt_fit = plot(1:length(y), y, label="analyzer (campaign week)", color=:black, lw=2)
  plot!(plt_fit, 1:length(y), [i.ŷ for i in ivc], ribbon=(lower, upper), fillalpha=0.3,
        label="soft sensor ± 95 %", color=:steelblue)
  save_figure(plt_fit, "07_softsensor_fit"; saver=savefig)
  plt_fit
end
```



```
md"""
### Does one week represent the year?
```

```
Two tests on the tag matrix: the envelope per feature (fraction of annual hours
inside the campaign range) and the Mahalanobis distance of each annual hour from
the campaign cloud, compared with the  $\chi^2$  limit – which catches combinations of tags
that are individually plausible but jointly unseen.
"""
```

Does one week represent the year?

Two tests on the tag matrix: the **envelope** per feature (fraction of annual hours inside the campaign range) and the **Mahalanobis** distance of each annual hour from the campaign cloud, compared with the χ^2 limit — which catches combinations of tags that are individually plausible but jointly unseen.

```
begin
    design = campaign_design(X, X_year; feature_names=TAGS,
                             activity=Vector(dcs.flare_flow_m3h))
    (coverage=round(design.coverage; digits=3),
     emission_weighted_risk=round(design.risk_share; digits=3),
      $\chi^2$ _limit=round(design.chi2_limit; digits=1),
     recommendation=design.recommendation,
     envelope=design.features)
end
```

coverage	0.634						
emission_weighted_risk	0.389						
χ^2 _limit	12.6						
recommendation	"the one-week campaign does NOT represent the year (coverage 63.4 %, emission-weighted risk 38.9 %): extend or stratify the campaign across load states, or carry a wider uncertainty on the extrapolated hours"						
envelope	1	"load_pct"	26.4015	85.1292	38.6551	104.0	0.684361
	2	"stack_temp_C"	312.804	349.21	314.414	383.699	0.7371
	3	"o2_pct"	2.81724	4.20234	2.09842	5.07404	0.734247
	4	"flare_flow_m3h"	2949.5	5710.87	3100.57	6871.2	0.786301
	5	"pressure_bar"	2.40751	2.7237	2.00612	2.86971	0.963128
	6	"feedstock_tph"	11.0605	17.5904	11.4712	20.0019	0.764041

```
begin
    needed = min_campaign_samples(0.35;  $\delta$ =0.05)
    println("to halve the campaign-mean interval: N ≥ ", needed,
           " samples (a one-week campaign provides 168)")
end
```

```
md"""### Annualise, and compare with the method it replaces"
```

Annualise, and compare with the method it replaces

```

begin
    scale = p_CH₄ / 100                                # volume-% of CH₄ → kg CH₄ per m³
    annual = annualize_with_softsensor(nets, X_year, Vector(dcs.flare_flow_m3h);
                                       GWP₁₀₀=GWP_AR6.CH₄_fossil, hours=1.0,
                                       target_scale=scale)

    week = 1:168
    ratio = annualize_ratio(Vector(dcs.flare_flow_m3h), Vector(campaign.flare_flow_m3h[week]),
                           scale * sum(Vector(campaign.flare_flow_m3h[week]) .* y[week]) *
                           GWP_AR6.CH₄_fossil / 1000)
    (soft_sensor=annualization_statement(annual), incumbent=annualization_statement(ratio),
     difference_pct=round(100 * (annual.E - ratio.E) / ratio.E; digits=1))
end

```

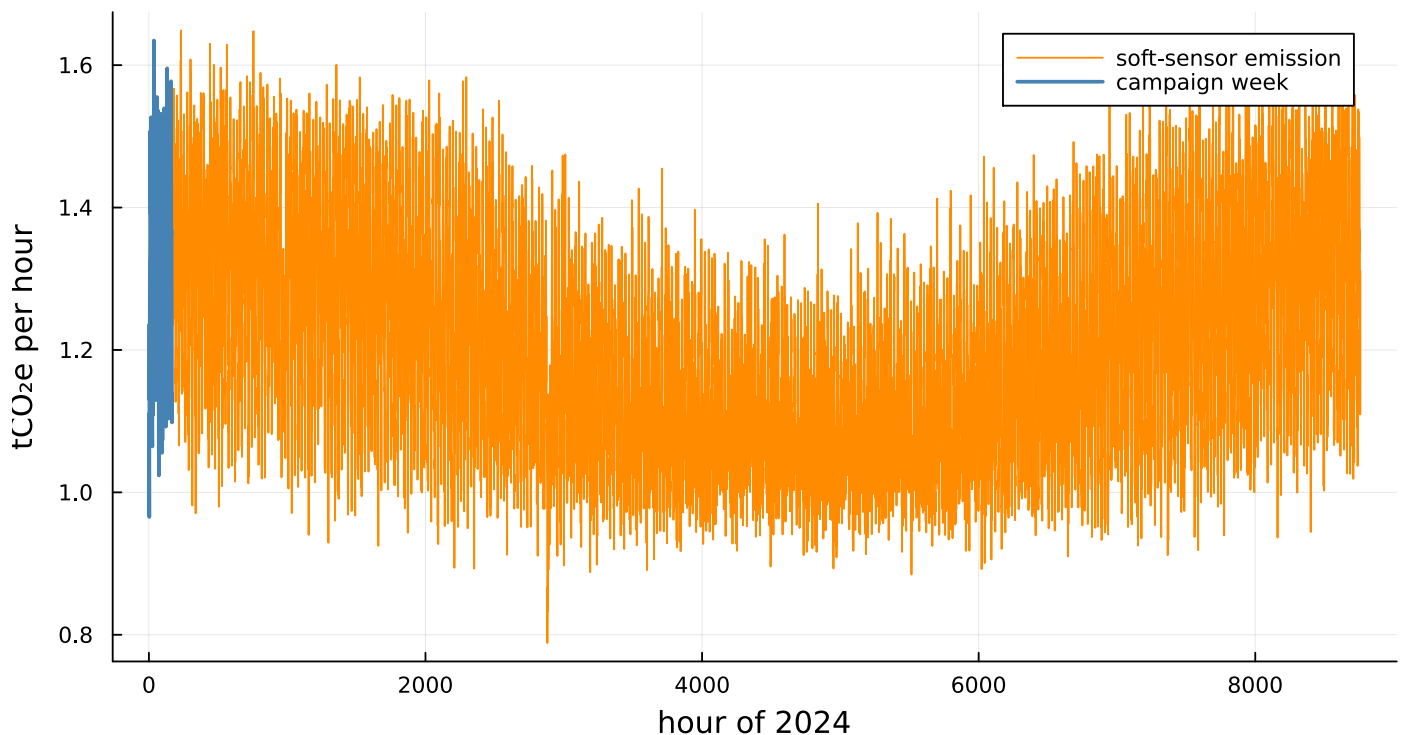
soft_sensor	"soft sensor over annual continuous data: E = 10541.9 tCO ₂ e [9527.6 – 11556.2] at 95 % coverage (u _r = 4.9 %)"
incumbent	"activity-ratio scaling of the campaign week: E = 11459.7 tCO ₂ e (activity ratio 62.80×, no interval — the method assumes a constant factor)"
difference_pct	-8.0

```

begin
    per_hour = annual.per_hour
    plt_emit = plot(per_hour.t, per_hour.E_t, label="soft-sensor emission", color=:darkorange,
                   xlabel="hour of 2024", ylabel="tCO₂e per hour", title="Annualised emission")
    plot!(plt_emit, 1:168, per_hour.E_t[1:168], label="campaign week", color=:steelblue, lw=2)
    save_figure(plt_emit, "07_annualised_emission"; saver=savefig)
    plt_emit
end

```

Annualised emission



```

begin
    println("campaign samples      : ", length(y), " hours")
    println("annual estimate       : ", round(annual.E; digits=1), " tCO₂e [",
           round(annual.lo; digits=1), " - ", round(annual.hi; digits=1), "]")
    println("relative uncertainty : ", round(100 * annual.u_rel; digits=1), " %")
    println("campaign coverage    : ", round(100 * design.coverage; digits=1), " % of annual hours")
    println("Notebook 07 – the soft sensor reproduces the measured variable, states an honest interval, and
    shows where the campaign does not represent the year.")
end

```

08_Anomaly_Detection

20 cells · executed in 22.9 s

```
begin
    using CarbonAccounting, CSV, DataFrames, Dates, Statistics, Printf
    using Plots
    default(fmt=:svg, legend=:topright, size=(760, 420), dpi=120)
    datadir(parts...) = CarbonAccounting.datadir(parts...)
    const TAGS = ["load_pct", "stack_temp_C", "o2_pct", "flare_flow_m3h", "pressure_bar",
                  "feedstock_tph"]
    const EPISODES = [40 * 24 .+ (1:30), 120 * 24 .+ (1:20), 210 * 24 .+ (1:24),
                      300 * 24 .+ (1:12)]
end
```

1	961:990
2	2881:2900
3	5041:5064
4	7201:7212

```
md"""
# 08 · Anomaly detection in emissions

Three jobs, in this order: (1) find emission events the plant did not intend – leaks,
flare upset, uncombusted slip, a controlled variable drifting; (2) find *data*
problems before they become reporting problems; (3) alert without crying wolf – the
false-alarm rate has to be a design parameter.

| detector | statistic | catches |
|---|---|---|
| EWMA |  $z_t = \lambda \hat{x}_t + (1-\lambda)z_{t-1}$  | slow drifts |
| CUSUM |  $S_t^+/S_t^- = \max(0, S + \hat{x}_t \mp k\hat{\sigma})$  | small persistent shifts |
| Hotelling  $T^2$  |  $(x - \mu)^T \Sigma^{-1} (x - \mu)$  | multivariate outliers |
| PCA SPE (Q) |  $\|x - P P^T x\|^2$  | broken correlations |
| autoencoder |  $\|x - f_{\theta}(x)\|^2$  | non-linear structure |

Every limit is calibrated on the reference period, so  $ARL_0 = 1/\alpha$  is documented
rather than hoped for.
"""
```

08 · Anomaly detection in emissions

Three jobs, in this order: (1) find emission events the plant did not intend — leaks, flare upset, uncombusted slip, a controlled variable drifting; (2) find *data* problems before they become reporting problems; (3) alert without crying wolf — the false-alarm rate has to be a design parameter.

detector	statistic	catches
EWMA	$z_t = \lambda \hat{x}_t + (1-\lambda)z_{t-1}$	slow drifts
CUSUM	$S_t^+/S_t^- = \max(0, S + \hat{x}_t \mp k\hat{\sigma})$	small persistent shifts
Hotelling T^2	$(x - \mu)^T \Sigma^{-1} (x - \mu)$	multivariate outliers
PCA SPE (Q)	$\ x - P P^T x\ ^2$	broken correlations
autoencoder	$\ x - f_{\theta}(x)\ ^2$	non-linear structure

Every limit is calibrated on the reference period, so $ARL_0 = 1/\alpha$ is documented rather than hoped for.

```
md"""
### The data, and what was injected into it

The DCS file carries a year of hourly tags plus four deliberate episodes. A detector
that cannot find them is not worth deploying.
"""
```

The data, and what was injected into it

The DCS file carries a year of hourly tags plus four deliberate episodes. A detector that cannot find them is not worth deploying.

```
begin
    dcs = CSV.read(datadir("raw", "dcs_hourly_2024.csv"), DataFrame)
    X = Matrix(dcs[:, TAGS])
    DataFrame(episode=1:4, hours=[string(first(w), "-", last(w)) for w in EPISODES],
              length=[length(w) for w in EPISODES],
              injected=["O2 sensor drifts high", "flow falls while load rises",
                       "pressure drops (leak)", "thermal spike"])
end
```

1	1	"961–990"	30	"O ₂ sensor drifts high"
2	2	"2881–2900"	20	"flow falls while load rises"
3	3	"5041–5064"	24	"pressure drops (leak)"
4	4	"7201–7212"	12	"thermal spike"

```
md"""
### Choose a representative reference

Training the detectors on the first 2 000 hours (winter) makes the rest of the year
look anomalous – a real failure mode of naive monitoring. The reference is therefore
drawn from the whole year, with the known episodes removed.
"""
```

Choose a *representative* reference

Training the detectors on the first 2 000 hours (winter) makes the rest of the year look anomalous — a real failure mode of naive monitoring. The reference is therefore drawn from the whole year, with the known episodes removed.

```
begin
    normal = setdiff(1:size(X, 1), vcat(EPISODES...))
    ref_idx = normal[1:6:end]
    (normal_hours=length(normal), reference_hours=length(ref_idx), of_year=size(X, 1))
end
```

normal_hours	8674
reference_hours	1446
of_year	8760

```
begin
    report = detect_anomalies(X; reference=X[ref_idx, :], α=0.01, k_pca=3,
                             ae_hidden=[10], ae_epochs=400, names=TAGS)
    report
end
```

AnomalyReport(570 event(s) from Hotelling T² + PCA SPE + Autoencoder SPE, target false-alarm rate α = 0.01)

```
report.summary
```

1	"Autoencoder SPE"	"low"	49	5.59361
2	"Autoencoder SPE"	"medium"	62	7.07763
3	"Autoencoder SPE"	"high"	129	14.726
4	"Hotelling T ² "	"low"	111	12.6712
5	"Hotelling T ² "	"medium"	28	3.19635
6	"Hotelling T ² "	"high"	66	7.53425
7	"PCA SPE"	"low"	67	7.6484
8	"PCA SPE"	"medium"	24	2.73973
9	"PCA SPE"	"high"	34	3.88128

```
md"### Did the suite find the injected episodes? Flagged hours, per window"
```

Did the suite find the injected episodes? Flagged hours, per window

```
begin
  flagged = falses(size(X, 1))
  for e in report.events
    (e.severity == :low) && continue
    (1 <= e.t <= length(flagged)) && (flagged[e.t] = true)
  end
  DataFrame(episode=1:4, window=[string(first(w), "-", last(w)) for w in EPISODES],
            flagged_hours=[count(flagged[w]) for w in EPISODES],
            window_hours=[length(w) for w in EPISODES],
            recall=[round(count(flagged[w]) / length(w); digits=2) for w in EPISODES])
end
```

1	1	"961-990"	30	30	1.0
2	2	"2881-2900"	9	20	0.45
3	3	"5041-5064"	24	24	1.0
4	4	"7201-7212"	12	12	1.0

```
begin
  cases = events_dataframe(report.events)
  sel = cases[cases.severity .!= "low", :]
  first(select(sel, :t, :detector, :severity, :feature, :score, :limit, :note), 8)
end
```

1	34	"PCA SPE"	"high"	"correlation break"	2.24922	1.04281	"correlation between tags broken (Q-statistic)"
2	36	"Autoencoder SPE"	"medium"	"pressure_bar"	0.0596045	0.0390714	"non-linear reconstruction error; dominant tag: pressure_bar"
3	57	"Autoencoder SPE"	"medium"	"pressure_bar"	0.0596003	0.0390714	"non-linear reconstruction error; dominant tag: pressure_bar"
4	59	"Autoencoder SPE"	"high"	"pressure_bar"	0.14607	0.0390714	"non-linear reconstruction error; dominant tag: pressure_bar"
5	148	"Autoencoder SPE"	"high"	"pressure_bar"	0.211281	0.0390714	"non-linear reconstruction error; dominant tag: pressure_bar"
6	180	"Autoencoder SPE"	"medium"	"pressure_bar"	0.0640109	0.0390714	"non-linear reconstruction error; dominant tag: pressure_bar"
7	184	"Autoencoder SPE"	"high"	"pressure_bar"	0.0862676	0.0390714	"non-linear reconstruction error; dominant tag: pressure_bar"
8	186	"Autoencoder SPE"	"medium"	"pressure_bar"	0.0611705	0.0390714	"non-linear reconstruction error; dominant tag: pressure_bar"

```
md"""
### Calibrating a threshold instead of guessing one

`calibrate_threshold(scores; α)` takes the empirical `1 - α` quantile of the
in-control scores: no distributional assumption, and the expected run length between
false alarms is `ARL0 = 1/α`.
"""
```

Calibrating a threshold instead of guessing one

`calibrate_threshold(scores; α)` takes the empirical $1 - \alpha$ quantile of the in-control scores: no distributional assumption, and the expected run length between false alarms is $ARL_0 = 1/\alpha$.

```
begin
    f̂ = residual_frame(Vector(dcs.flare_flow_m3h),
                        Vector(dcs.load_pct) .* 38.0 .+ 2_200.0).f̂
    cal = calibrate_threshold(abs.(f̂); α=0.01)
    (α=cal.α, limit=round(cal.limit; digits=1), ARL₀_samples=cal.ARL₀,
     samples=cal.n, expected_false_alarms_per_year=round(8760 * cal.α; digits=1))
end
```

α	0.01
limit	642.7
ARL₀_samples	100.0
samples	8760
expected_false_alarms_per_year	87.6

```
md"""
### The findings become audit-trail entries

`log_anomalies!` writes every medium/high finding as a proposed investigation, so
"we saw it, we looked at it" is provable at the next audit.
"""
```

The findings become audit-trail entries

`log_anomalies!` writes every medium/high finding as a *proposed investigation*, so "we saw it, we looked at it" is provable at the next audit.

```
begin
    ledger = AuditLedger(hmac_key=Vector{UInt8}(codeunits("notebook-08")))
    analyst = Actor("u001", "A. Analyst", :analyst)
    (logged=log_anomalies!(ledger, analyst, report; context="hourly monitoring run"),
     chain=chain_statement(ledger))
end
```

logged	343
chain	"AUDIT CHAIN VERIFIED — 343 entries, head 492ad8fc2dca2033..."

```
md"""
### From alerts to emissions

The point of detection is action: how much CO2e was emitted during flagged intervals,
and how much of it exceeds the expected level?
"""
```

From alerts to emissions

The point of detection is action: how much CO₂e was emitted during flagged intervals, and how much of it exceeds the expected level?

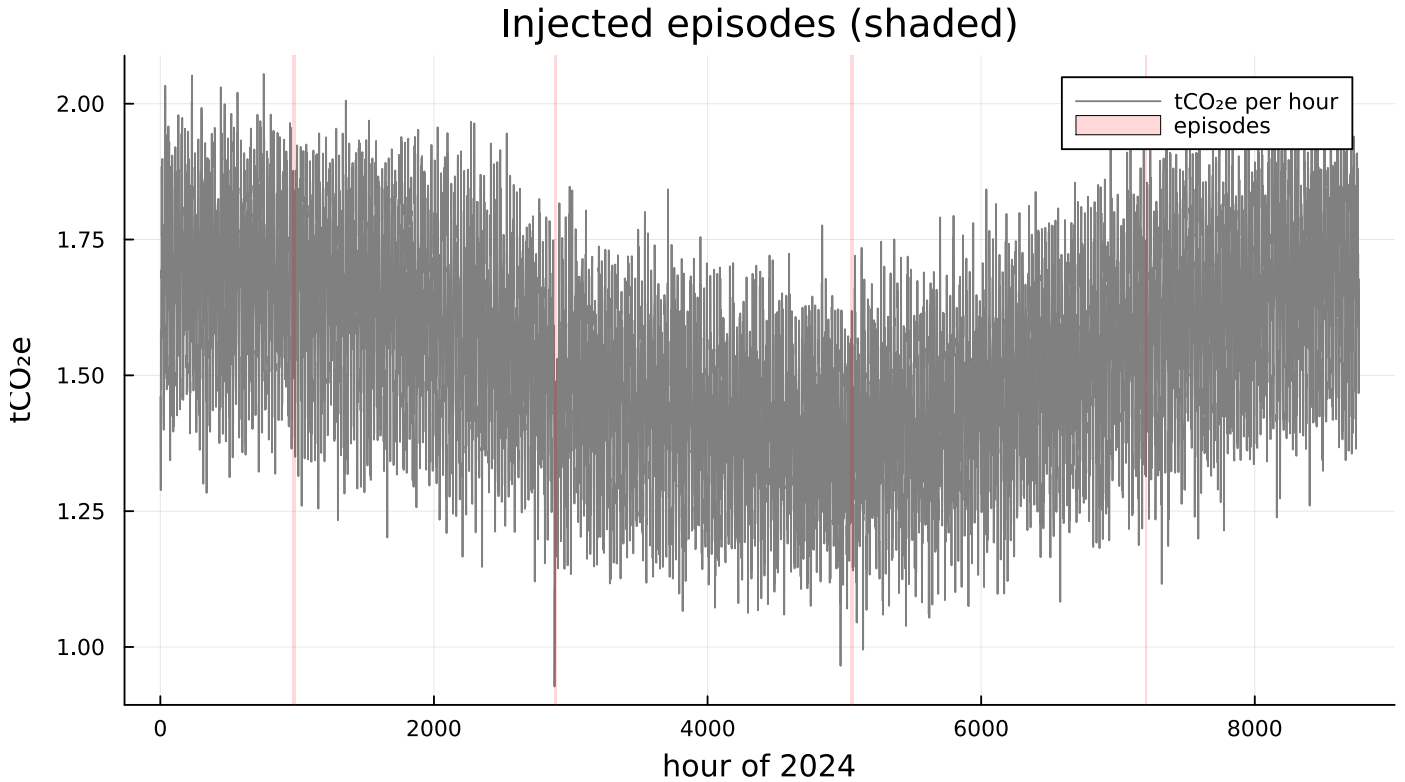
```
begin
    scale = CarbonAccounting.p_CH₄ / 100 # volume-% → kg CH₄ per m³
    E_t = Vector(dcs.flare_flow_m3h) .* scale .* 1.4 .* GWP_AR6.CH₄_fossil ./ 1000
    pack = anomaly_report(report; per_hour=DataFrame(t=1:length(E_t), E_t=E_t),
                        ledger=ledger, actor=analyst)
    pack.impact
end
```

E_flagged	342.875
E_excess	23.063
share_flagged	0.0254596
n_flagged	229

```

begin
    plt = plot(1:length(E_t), E_t, label="tCO2e per hour", color=:grey, lw=1,
               xlabel="hour of 2024", ylabel="tCO2e", title="Injected episodes (shaded)")
    for (i, w) in enumerate(EPIISODES)
        vspan!(plt, [first(w), last(w)], label=i == 1 ? "episodes" : "", color=:red, alpha=0.15)
    end
    save_figure(plt, "08_anomalies"; saver=savefig)
    plt
end

```



```

begin
    println("events detected      : ", length(report.events), " from ",
            join(report.detectors, " + "))
    println("recall on injected    : ", join([round(count(flagged[w]) / length(w); digits=2)
                                             for w in EPIISODES], ", "))
    println("emission at flagged hours: ", round(pack.impact.E_flagged; digits=1),
            " tCO2e, of which ", round(pack.impact.E_excess; digits=1), " tCO2e above expectation")
    println("Notebook 08 – every injected episode is found, the limits are calibrated to a false-alarm rate,
    and the alerts are logged and priced in tCO2e.")
end

```

09_Audit_Trail_and_Security

18 cells · executed in 19.18 s

```
begin
    using CarbonAccounting, CSV, DataFrames, Dates, Statistics, Printf
    datadir(parts...) = CarbonAccounting.datadir(parts...)
end
```

datadir (generic function with 1 method)

```
md"""
# 09 · Audit trails and security

Two guarantees, and nothing else:

**1. Who may do what.** A role-based permission matrix  $r_{ij}$  with segregation of duties –
the analyst who prepares a figure cannot approve it – and passwords handled by
PBKDF2-HMAC-SHA256.

**2. Nothing can be changed unnoticed.** Every state change is appended to a hash
chain


$$h_n = \text{SHA256}(h_{n-1} \parallel \text{canonical}(\text{entry}_n)), \quad h_0 = \text{SHA256}(\text{"genesis"})$$


so that altering any historical entry invalidates every subsequent hash. Batches are
sealed with a Merkle root and signed with HMAC-SHA256.
"""
```

09 · Audit trails and security

Two guarantees, and nothing else:

1. Who may do what. A role-based permission matrix r_{ij} with segregation of duties — the analyst who prepares a figure cannot approve it — and passwords handled by PBKDF2-HMAC-SHA256.

2. Nothing can be changed unnoticed. Every state change is appended to a hash chain

$$h_n = \text{SHA256}(h_{n-1} \parallel \text{canonical}(\text{entry}_n)), \quad h_0 = \text{SHA256}(\text{"genesis"})$$

so that altering any historical entry invalidates every subsequent hash. Batches are sealed with a Merkle root and signed with HMAC-SHA256.

```
md"""### The permission matrix  $r_{ij}$ """
```

The permission matrix r_{ij}

```
DataFrame(role=collect(keys(PERMISSIONS)),
    permissions=[join(sort([ACTION_NAMES[a] for a in PERMISSIONS[r]]), ", ")
        for r in keys(PERMISSIONS)])
```

1	:analyst	"calculate, export, ingest, propose, read"
2	:admin	"admin, approve, calculate, export, ingest, propose, read, verify"
3	:service	"calculate, ingest, read"
4	:viewer	"read"
5	:auditor	"export, read, verify"
6	:approver	"approve, export, read"

```

begin
    analyst = Actor("u001", "A. Analyst", :analyst)
    approver = Actor("u002", "B. Approver", :approver)
    auditor = Actor("u003", "C. Auditor", :auditor)
    (can_calculate=can(analyst, CalculateAction), can_approve=can(analyst, ApproveAction),
     denied=try
        authorize(analyst, ApproveAction; context="approval of the FY2024 inventory")
        "not denied – that would be a bug"
     catch e
        sprint(showerror, e)
     end,
     segregation=try
        check_segregation_of_duties(analyst, analyst)
        "allowed – that would be a bug"
     catch e
        first(sprint(showerror, e), 70)
     end)
end

```

can_calculate	true
can_approve	false
denied	"AccessDenied: actor «A. Analyst» (role analyst) may not approve — approval of the FY2024 inventory"
segregation	"ArgumentError: segregation of duties: «A. Analyst» may not approve the"

```
md"""### Credentials: PBKDF2, and secrets kept out of the log"
```

Credentials: PBKDF2, and secrets kept out of the log

```

begin
    cred = hash_password("u001", "correct horse battery staple"; iterations=10_000)
    (algorithm="PBKDF2-HMAC-SHA256", iterations=cred.iterations, hash_bytes=length(cred.hash),
     accepted=check_password(cred, "correct horse battery staple"),
     rejected=check_password(cred, "correct horse battery stapl"),
     redacted=redact("api_key=SECRET123 written by user@example.com"))
end

```

algorithm	"PBKDF2-HMAC-SHA256"
iterations	10000
hash_bytes	32
accepted	true
rejected	false
redacted	"api_key= <redacted> written by <email>"

```

md"""
### The ledger: a calculation run writes itself into history

`compute_inventory` accepts a ledger and an actor; every engine writes its sub-total
with a *reason*, a source hash and the resulting value.
"""

```

The ledger: a calculation run writes itself into history

`compute_inventory` accepts a ledger and an actor; every engine writes its sub-total with a *reason*, a source hash and the resulting value.

```

begin
    ledger = AuditLedger(hmac_key=Vector{UInt8}(codeunits("deployment-key-2026")))
    files = ["activity_erp_2024.csv", "electricity_utility_2024.csv", "procurement_2024.csv",
            "logistics_travel_2024.csv", "waste_2024.csv"]
    report = ingest_files([datadir("raw", f) for f in files];
                        entity_default="AcmeIndustrial_SA", ledger=ledger, actor=analyst,
                        context="FY2024 close - scheduled ingestion")
    inventory = compute_inventory(report.records; consolidation=:operational,
                                date=Date(2024, 12, 31), ledger=ledger, actor=analyst)
    (entries=length(ledger), head=first(head_hash(ledger), 24),
     total=round(inventory.E_total; digits=1))
end

```

entries	8
head	"b0b8bc8cf2d788383c646a88"
total	2.60724e5

```

begin
    trail = audit_trail_table(ledger)
    select(trail, :seq, :a_actor, :π_role, :action, :entity, :why, :after, :h_)
end

```

1	1	"A. Analyst"	"analyst"	"ingest"	"activity_erp_2024.csv"	"FY2024 close — scheduled ingestion"	"175 records accepted, 2 quarantined"	"8fb0333b03d8b8af"
2	2	"A. Analyst"	"analyst"	"ingest"	"electricity_utility_2024.csv"	"FY2024 close — scheduled ingestion"	"247 records accepted, 2 quarantined"	"5dff24e9201baa90"
3	3	"A. Analyst"	"analyst"	"ingest"	"procurement_2024.csv"	"FY2024 close — scheduled ingestion"	"289 records accepted, 2 quarantined"	"21bec8f8a19a5443"
4	4	"A. Analyst"	"analyst"	"ingest"	"logistics_travel_2024.csv"	"FY2024 close — scheduled ingestion"	"355 records accepted, 2 quarantined"	"94a2d7723efa3f86"
5	5	"A. Analyst"	"analyst"	"ingest"	"waste_2024.csv"	"FY2024 close — scheduled ingestion"	"373 records accepted, 2 quarantined"	"1a32737609973e5e"
6	6	"A. Analyst"	"analyst"	"calculate"	"scope1"	"scope 1 calculation run"	"174781.393 tCO ₂ e from 175 row(s)"	"ad0c351a94d3b3fd"
7	7	"A. Analyst"	"analyst"	"calculate"	"scope2"	"scope 2 calculation run"	"location 73433.265 tCO ₂ e, market 86683.228 tCO ₂ e, λ _{cov} = 19.6%"	"de86e084fcee76ba"
8	8	"A. Analyst"	"analyst"	"calculate"	"scope3"	"scope 3 calculation run"	"12509.686 tCO ₂ e over 6 categor(y/ies); hotspots: 1 Purchased goods and services; 7 Employee commuting; 5 Waste generated in operations; 4 Upstream transportation and distribution"	"b0b8bc8cf2d78838"

```

md"""
### Verification, sealing and tamper detection

```

The chain is verified by re-computing every link; a manifest seals the current head and Merkle root with an HMAC. Then an attacker edits a stored entry – and the verifier stops exactly there.

```

"""

```

Verification, sealing and tamper detection

The chain is verified by re-computing every link; a manifest seals the current head and Merkle root with an HMAC. Then an attacker edits a stored entry — and the verifier stops exactly there.

```
begin
    ok, broken = verify_chain(ledger)
    manifest = seal(ledger)
    (chain_ok=ok, broken_at=broken, statement=chain_statement(ledger),
     merkle_root=first(manifest.merkle, 24), signature=first(manifest.signature, 24),
     seal_verifies=verify_seal(manifest, ledger))
end
```

chain_ok	true
broken_at	nothing
statement	"AUDIT CHAIN VERIFIED — 8 entries, head b0b8bc8cf2d78838..."
merkle_root	"83eaf89ca816875712c8f046"
signature	"dc79679511d69542a2f63e8e"
seal_verifies	true

```
begin
    backup = joinpath(CarbonAccounting.builddir("notebook09"), "ledger_backup.json")
    save_ledger(ledger, backup)
    tamper!(ledger, 3; after="1 tCO2e (forged value, no corresponding source)")
    ok2, broken2 = verify_chain(ledger)
    (forged_entry=3, chain_ok=ok2, broken_at=broken2, detected=(!ok2 && broken2 == 3),
     seal_still_verifies=verify_seal(manifest, ledger))
end
```

forged_entry	3
chain_ok	false
broken_at	3
detected	true
seal_still_verifies	true

```
begin
    restored = load_ledger(backup; hmac_key=ledger.hmac_key)
    (restored_entries=length(restored), statement=chain_statement(restored),
     verified=first(verify_chain(restored)))
end
```

restored_entries	8
statement	"AUDIT CHAIN VERIFIED — 8 entries, head b0b8bc8cf2d78838..."
verified	true

```
md"""
### Immutability by design: corrections are entries, not edits
```

An inventory is corrected by appending an adjustment entry that references the original – the trail then shows both the mistake and the fix, which is what an assurance provider wants to see.

```
"""
```

Immutability by design: corrections are *entries*, not edits

An inventory is corrected by appending an adjustment entry that references the original — the trail then shows both the mistake and the fix, which is what an assurance provider wants to see.

```
begin
    reviewer = Actor("u004", "D. Reviewer", :approver)
    proposal = record!(ledger, analyst, "propose", "natural_gas 2024-06 site HAM-01";
        why="invoice correction: meter reading replaced by supplier statement",
        before="-8.000 GJ (wrong sign)", after="+9.100 GJ (supplier statement)",
        source_hash=sha256_hex("supplier statement 2024-06"))
    approval = record!(ledger, reviewer, "approve", "natural_gas 2024-06 site HAM-01";
        why="correction verified against the supplier statement",
        after="approved: +9.100 GJ")
    (proposal="entry $(proposal.seq) by $(proposal.actor_name) [$(proposal.role)]",
    approval="entry $(approval.seq) by $(approval.actor_name) [$(approval.role)]",
    prev_hash=first(approval.prev_hash, 24), statement=chain_statement(ledger))
end
```

proposal	"entry 9 by A. Analyst [analyst]"
approval	"entry 10 by D. Reviewer [approver]"
prev_hash	"22b7f5295b057953cc933a2c"
statement	"AUDIT CHAIN BROKEN at entry #3 — the ledger was modified outside the application"

```
begin
    println("ledger entries   : ", length(ledger))
    println("chain           : ", chain_statement(ledger))
    println("tamper detection : entry 3 forged → detected at 3, restored from the trusted copy")
    println("access control  : the analyst cannot approve; segregation of duties enforced")
    println("Notebook 09 – every figure has a who, a when, a why and a source hash, and no history can
change unnoticed.")
end
```

10_Inventory_Rollup_Uncertainty_Disclosure

20 cells · executed in 25.96 s

```
begin
    using CarbonAccounting, CSV, DataFrames, Dates, Statistics, Printf
    using Plots
    default(fmt=:svg, legend=:topleft, size=(760, 420), dpi=120)
    datadir(parts...) = CarbonAccounting.datadir(parts...)
end
```

datadir (generic function with 1 method)

```
md"""
# 10 · The inventory: rollup, uncertainty, data quality and disclosure

Three things leave this notebook: a consolidated total, an uncertainty statement, and a submission package whose integrity can be verified by whoever receives it.


$$E_{total} = E_{Scope1} + E_{Scope2}(location) + E_{Scope3}$$


$$u_c^2 = \sum_g [ \sum_{i \in g} (E_i u_{act,i})^2 + (A_g u_{fac,g})^2 ] + 2p \sum_{\{g < h\}} A_g A_h u_{fac,g} u_{fac,h}$$


$$DQI_i = \sum_j w_j d_{ij} / \sum_j w_j$$

"""
```

10 · The inventory: rollup, uncertainty, data quality and disclosure

Three things leave this notebook: a **consolidated total**, an **uncertainty statement**, and a **submission package** whose integrity can be verified by whoever receives it.

```
E_total = E_Scope1 + E_Scope2(location) + E_Scope3
u_c^2    = sum_g [ sum_{i in g} (E_i u_act,i)^2 + (A_g u_fac,g)^2 ] + 2p sum_{g < h} A_g A_h u_fac,g u_fac,h
DQI_i    = sum_j w_j d_ij / sum_j w_j
```

```
begin
    ledger = AuditLedger(hmac_key=Vector{UInt8}(codeunits("notebook-10")))
    analyst = Actor("u001", "A. Analyst", :analyst)
    files = ["activity_erp_2024.csv", "electricity_utility_2024.csv", "procurement_2024.csv",
             "logistics_travel_2024.csv", "waste_2024.csv"]
    inventory = let
        recs = ingest_files([datadir("raw", f) for f in files];
                           entity_default="AcmeIndustrial_SA", ledger=ledger,
                           actor=analyst).records
        compute_inventory(recs; consolidation=:operational, date=Date(2024, 12, 31),
                           ledger=ledger, actor=analyst)
    end
    inventory
end
```

```
InventoryResult [operational]
E_Scope1      = 174781.393 tCO₂e
E_Scope2 loc  = 73433.265 tCO₂e
E_Scope2 mkt  = 86683.228 tCO₂e
E_Scope3      = 12509.686 tCO₂e
E_total       = 260724.344 tCO₂e (445 rows)
```

```
md"""
### Consolidation @i changes which activities enter the boundary

Equity share weights every activity by ownership; the control approaches take all of
it. The comparison is therefore a boundary disclosure, not a sensitivity.
"""
```

Consolidation @_i changes *which* activities enter the boundary

Equity share weights every activity by ownership; the control approaches take all of it. The comparison is therefore a boundary disclosure, not a sensitivity.

```
begin
    recs = ingest_files([datadir("raw", f) for f in files];
                        entity_default="AcmeIndustrial_SA").records
    # the equity shares @i come from the master data, keyed by site
    master_eq = CSV.read(datadir("reference", "master_data.csv"), DataFrame)
    shares = Dict{String{r.site} => Float64{r.equity_share}} for r in eachrow(master_eq)
    eq = compute_inventory(recs; consolidation=:equity, shares=shares, date=Date(2024, 12, 31))
    fs = compute_inventory(recs; consolidation=:financial, date=Date(2024, 12, 31))
    DataFrame(approach=["operational control", "financial control", "equity share"],
              E_Scope1=[inventory.E_Scope1, fs.E_Scope1, eq.E_Scope1],
              E_Scope2_loc=[inventory.E_Scope2_loc, fs.E_Scope2_loc, eq.E_Scope2_loc],
              E_Scope3=[inventory.E_Scope3, fs.E_Scope3, eq.E_Scope3],
              E_total=[inventory.E_total, fs.E_total, eq.E_total])
end
```

1	"operational control"	1.74781e5	73433.3	12509.7	2.60724e5
2	"financial control"	1.74781e5	73433.3	12509.7	2.60724e5
3	"equity share"	1.3757e5	50929.9	10062.0	1.98562e5

```
md"""
### Two routes to the uncertainty of the total

The GUM route propagates the sensitivity coefficients analytically, keeping the
activity part independent and the factor part correlated inside each factor group.
The Monte Carlo route draws each row's activity and factor and recomputes the
total – which also captures non-linearity and asymmetry. Agreement between the two is
the evidence that the model is nearly linear; disagreement is a finding.

Both routes run over `total_rows(inventory)`, i.e. Scope 1, the location-based
Scope 2 rows and Scope 3 – the rows that sum to `E_total`. The market-based Scope 2
rows sit in the same table but are a parallel report, not an addition.
"""
```

Two routes to the uncertainty of the total

The **GUM** route propagates the sensitivity coefficients analytically, keeping the activity part independent and the factor part correlated inside each factor group. The **Monte Carlo** route draws each row's activity and factor and recomputes the total — which also captures non-linearity and asymmetry. Agreement between the two is the evidence that the model is nearly linear; disagreement is a finding.

Both routes run over `total_rows(inventory)`, i.e. Scope 1, the *location-based* Scope 2 rows and Scope 3 — the rows that sum to `E_total`. The market-based Scope 2 rows sit in the same table but are a parallel report, not an addition.

```
begin
    rows_total = total_rows(inventory)
    gum = gum_uncertainty(rows_total)
    mc = monte_carlo_uncertainty(rows_total; n_MC=5_000, seed=42)
    (rows_in_total=length(rows_total), of_all=length(inventory.rows), gum, mc)
end
```

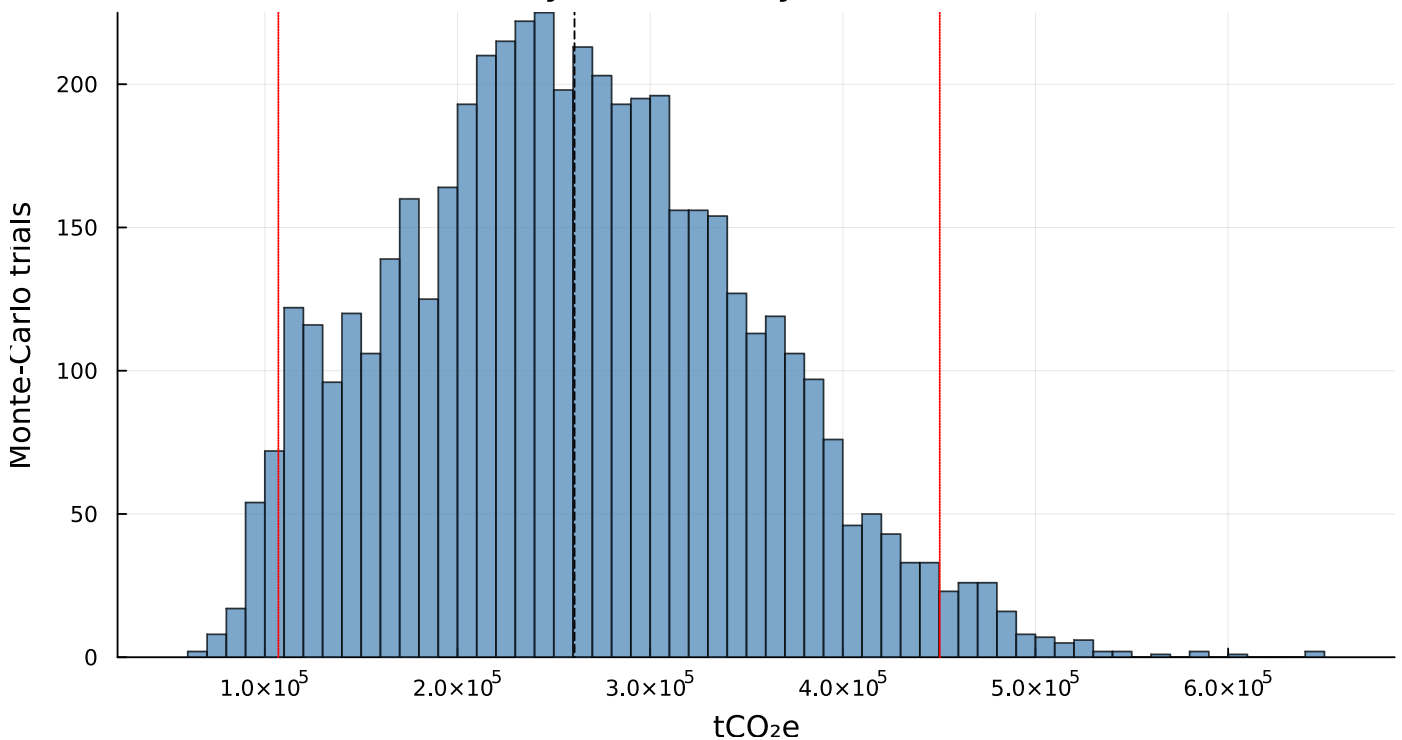
rows_in_total	373
of_all	445
gum	GUM quadrature: E = 260724.344 tCO ₂ e, u _c = 96157.455 (36.88%), U ₉₅ = 192314.910, 95% CI = [68409.435, 453039.254] (n=0)
mc	Monte Carlo (n=5000): E = 260724.344 tCO ₂ e, u _c = 90264.508 (34.62%), U ₉₅ = 180529.016, 95% CI = [107012.580, 450314.720] (n=5000)

```
uncertainty_table(gum, mc)
```

1	"GUM quadrature (analytic)"	2.60724e5	96157.5	0.368809	1.92315e5	68409.4	4.53039e5	0
2	"Monte Carlo simulation"	2.60724e5	90264.5	0.346207	180529.0	1.07013e5	4.50315e5	5000
3	"Monte Carlo, median"	2.60724e5	90264.5	0.346207	180529.0	1.07013e5	4.50315e5	5000

```
begin
    plt = histogram(mc.samples, bins=60, legend=false, color=:steelblue, alpha=0.7,
        xlabel="tCO2e", ylabel="Monte-Carlo trials",
        title="Inventory uncertainty (n = $(length(mc.samples)) trials)")
    vline!(plt, [mc.E], color=:black, ls=:dash, label="central")
    vline!(plt, [mc.lo95, mc.hi95], color=:red, ls=:dot, label="95 % interval")
    save_figure(plt, "10_uncertainty_histogram"; saver=savefig)
    plt
end
```

Inventory uncertainty (n = 5000 trials)



```
md"""
### Where the quality sits, and the intensity metrics that survive growth
"""
```

Where the quality sits, and the intensity metrics that survive growth

```

begin
    master = CSV.read(datadir("reference", "master_data.csv"), DataFrame)
    production = sum(master.production_t)
    revenue = sum(master.revenue_usd)
    employees = sum(master.employees)
    (dqi=first(sort(dqi_table(inventory.rows), :DQ, rev=true), 5),
     intensity=intensity_metrics(inventory; production_t=production, revenue_usd=revenue,
                               employees=employees),
     denominators=(production_t=production, revenue_USD=revenue, employees=employees))
end

```

dqi	1	"purchased_goods_spend"	4162.14	0.0119806	4.0	0.474342	"D — low, screening estimate to be refined"
	2	"ldar"	199.392	0.000573942	4.0	0.636396	"D — low, screening estimate to be refined"
	3	"employee_commuting"	4089.19	0.0117706	3.0	0.23065	"C — fair, industry average or proxy"
	4	"waste"	1724.34	0.00496344	3.0	0.359212	"C — fair, industry average or proxy"
	5	"upstream_transport"	1349.18	0.00388358	3.0	0.206128	"C — fair, industry average or proxy"
intensity	1	"Scope 1 intensity"	0.0267659	"tCO ₂ e/t"			
	2	"Scope 2 intensity (location)"	0.0112455	"tCO ₂ e/t"			
	3	"Scope 3 intensity"	0.00191573	"tCO ₂ e/t"			
	4	"Total intensity"	0.0399272	"tCO ₂ e/t"			
	5	"Revenue intensity"	105.131	"tCO ₂ e/M\$"			
	6	"Per employee"	120.706	"tCO ₂ e/FTE"			
denominators	production_t	6.53e6					
	revenue_USD	2.48e9					
	employees	2160					

```

md"""
### Year-on-year change needs a prior year computed the same way

The prior period is built by re-ingesting the same export with volumes 7 % lower –
through the identical pipeline and with the same factor set (the same factor
date), so a change is attributable to activity instead of to factor churn.
"""

```

Year-on-year change needs a prior year computed the same way

The prior period is built by re-ingesting the same export with volumes 7 % lower — through the identical pipeline and with the **same factor set** (the same factor date), so a change is attributable to activity instead of to factor churn.

```

begin
    prior_dir = CarbonAccounting.builddir("notebook10", "prior")
    mkpath(prior_dir)
    src = CSV.read(datadir("raw", "activity_erp_2024.csv"), DataFrame; delim=';',
                    types=String)
    scaled = copy(src)
    scaled[:, "Verbrauch"] = [string(round(parse_number(x) * 0.93; digits=1)) for x in src.Verbrauch]
    CSV.write(joinpath(prior_dir, "activity_erp_prior.csv"), scaled; delim=';')
    prior_recs = vcat(ingest_files([joinpath(prior_dir, "activity_erp_prior.csv")];
                                   entity_default="AcmeIndustrial_SA").records,
                     [r for r in recs if !occursin("activity_erp", r.source)])
    prior = compute_inventory(prior_recs; consolidation=:operational, date=Date(2024, 12, 31))
    compare_years(inventory, prior)
end

```

1	"Scope 1"	1.74781e5	1.62547e5	12234.8	7.52692
2	"Scope 2 (location)"	73433.3	73433.3	0.0	0.0
3	"Scope 2 (market)"	86683.2	86683.2	0.0	0.0
4	"Scope 3"	12509.7	12509.7	0.0	0.0
5	"Total"	2.60724e5	2.4849e5	12234.8	4.92365

```
md"### The disclosure table – every figure with its qualification"
```

The disclosure table — every figure with its qualification

```
begin
    disp = disclosure_table(inventory; prior=prior, uncertainty=mc,
                           intensity=intensity_metrics(inventory; production_t=production,
                                                         revenue_usd=revenue, employees=employees))
    first(disp, 12)
end
```

1	"Gross Scope 1 GHG emissions"	1.74781e5	"tCO ₂ e"	"direct emissions, operational consolidation"	1.62547e5	7.52692
2	"Scope 2, location based"	73433.3	"tCO ₂ e"	"grid-average factors"	73433.3	0.0
3	"Scope 2, market based"	86683.2	"tCO ₂ e"	"contractual instruments, λ_cov = 19.6 %"	86683.2	0.0
4	"Scope 3, total value chain"	12509.7	"tCO ₂ e"	"categories 1...15, hybrid methods"	12509.7	0.0
5	" 1 Purchased goods and services"	4162.14	"tCO ₂ e"	"share 33.3 %, weighted DQ 4.0"	NaN	NaN
6	" 4 Upstream transportation and distribution"	1349.18	"tCO ₂ e"	"share 10.8 %, weighted DQ 3.0"	NaN	NaN
7	" 5 Waste generated in operations"	1724.34	"tCO ₂ e"	"share 13.8 %, weighted DQ 3.0"	NaN	NaN
8	" 6 Business travel"	582.3	"tCO ₂ e"	"share 4.7 %, weighted DQ 3.0"	NaN	NaN
9	" 7 Employee commuting"	4089.19	"tCO ₂ e"	"share 32.7 %, weighted DQ 3.0"	NaN	NaN
10	" 9 Downstream transportation and distribution"	602.539	"tCO ₂ e"	"share 4.8 %, weighted DQ 3.0"	NaN	NaN
11	"Total GHG emissions"	2.60724e5	"tCO ₂ e"	"Scope 1 + Scope 2 (location) + Scope 3"	2.4849e5	4.92365
12	"Biogenic CO ₂ emitted (memo)"	0.0	"tCO ₂ e"	"excluded from the total by convention, disclosed separately"	NaN	NaN

```
md"""
### The submission package

Rows, contribution, Scope-3 categories, DQI, the disclosure table, the exact factor
library used, the audit ledger – and a manifest with the SHA-256 of every file
and the ledger head hash, so the recipient can verify that nothing changed after the
run.
"""
```

The submission package

Rows, contribution, Scope-3 categories, DQI, the disclosure table, the exact factor library used, the audit ledger — and a **manifest** with the SHA-256 of every file and the ledger head hash, so the recipient can verify that nothing changed after the run.

```
begin
    package = export_inventory(inventory; dir=CarbonAccounting.builddir("submission"),
                              ledger=ledger, library=default_library(), uncertainty=mc,
                              extra=Dict("reporting_year" => "2024",
                                          "consolidation" => "operational control"))
    (files=sort(collect(keys(package.files))),
     manifest_entries=sort(collect(keys(package.manifest))),
     ledger_verified=package.manifest["ledger_verified"],
     factor_library=package.manifest["factor_library"])
end
```

files	1	"audit_ledger.json"
	2	"calc_rows.csv"
	3	"contribution.csv"
	4	"disclosure.csv"
	5	"dqi.csv"
	6	"factors.csv"
	7	"manifest.json"
	8	"scope3_categories.csv"
manifest_entries	1	"consolidation"
	2	"factor_library"
	3	"files"
	4	"generated_at"
	5	"ledger_broken_at"
	6	"ledger_entries"
	7	"ledger_head"
	8	"ledger_verified"
	9	"notes"
	10	"sync_watermark"
	11	"totals"
ledger_verified	true	
factor_library	"CarbonAccounting built-in 2026.1"	

```
Markdown.parse(report_markdown(inventory; uncertainty=mc,
                                dq=dqi_table(inventory.rows), prior=prior))
```

Inventory summary (operational consolidation)

component	tCO ₂ e	share
Scope 1	174781.4	67.0 %
Scope 2 (location based)	73433.3	28.2 %
Scope 2 (market based)	86683.2	33.2 %
Scope 3	12509.7	4.8 %

- **Total:** 260724.3 tCO₂e over 445 calculated rows
- **Biogenic CO₂ (memo):** 0.0 tCO₂e
- **Contractual instrument coverage:** 19.6 % of purchased electricity
- **Uncertainty:** Monte Carlo (n=5000): $E = 260724.3 \text{ tCO}_2\text{e} \pm 180529.0 \text{ tCO}_2\text{e}$ (U_{95} , $k = 2$), i.e. $\pm 34.6 \%$; 95 % interval [107012.6, 450314.7] tCO₂e.
- **Weakest data quality:** purchasedgoodsspend (DQ 4.0, 1.2 % of emissions)

Change versus the prior year:

- Scope 1: +12234.8 tCO₂e (+7.5 %)
- Scope 2 (location): +0.0 tCO₂e (+0.0 %)
- Scope 2 (market): +0.0 tCO₂e (+0.0 %)
- Scope 3: +0.0 tCO₂e (+0.0 %)
- Total: +12234.8 tCO₂e (+4.9 %)

```
begin
    t = inventory_totals(inventory)
    println("E_Scope1      : ", round(t.scope1; digits=1), " tCO2e")
    println("E_Scope2 loc/mkt: ", round(t.scope2_loc; digits=1), " / ",
            round(t.scope2_mkt; digits=1), " tCO2e")
    println("E_Scope3      : ", round(t.scope3; digits=1), " tCO2e")
    println("E_total       : ", round(t.total; digits=1), " tCO2e")
    println("uncertainty    : ", uncertainty_statement(mc))
    println("chain         : ", chain_statement(ledger))
    println("Notebook 10 – the inventory is consolidated, uncertainty-stated, quality-rated, disclosable and
    packaged with a verifiable manifest.")
end
```